ZT-SDN: An ML-Powered Zero-Trust Architecture for Software-Defined Networks

CHARALAMPOS KATSIS, Department of Computer Science, Purdue University, West Lafayette, United States
ELISA BERTINO, Department of Computer Science, Purdue University, West Lafayette, United States

Zero Trust (ZT) is a security paradigm aiming to curtail an attacker's lateral movements within a network by implementing least-privilege and per-request access control policies. However, its widespread adoption is hindered by the difficulty of generating proper rules owing to the lack of detailed knowledge of communication requirements and the characteristic behaviors of communicating entities under benign conditions. Consequently, manual rule generation becomes cumbersome and error prone. To address these problems, we propose *ZT-SDN*, an automated framework for learning and enforcing network access control in Software-Defined Networks (SDNs). ZT-SDN collects data from the underlying network and models the network "transactions" performed by communicating entities as graphs. The nodes represent entities, whereas the directed edges represent transactions identified by different protocol stacks observed. It uses novel unsupervised learning approaches to extract transaction patterns directly from the network data, such as the allowed protocol stacks and port numbers and data transmission behavior. Finally, ZT-SDN uses an innovative approach to generate correct access control rules and infer strong associations between them, allowing proactive rule deployment in forwarding devices. We show the framework's efficacy in detecting abnormal network accesses and abuses of permitted flows in changing network conditions with real network datasets. Additionally, we showcase ZT-SDN's scalability and the network's performance when applied in an SDN environment.

CCS Concepts: • **Networks** → *Network management*; **Network monitoring**; **Firewalls**; **Programmable networks;**

Additional Key Words and Phrases: SDN, zero-trust, network access control, firewall rules

ACM Reference Format:
Charalampos Katsis and Elisa Bertino. 2025. ZT-SDN: An ML-Powered Zero-Trust Architecture for Software-Defined Networks. *ACM Trans. Priv. Sec.* 28, 2, Article 23 (February 2025), 35 pages. https://doi.org/10.1145/3712262

1 Introduction

Zero Trust (ZT) [45] is a security paradigm that can significantly enhance network security. With ZT, the traditional perimeter-based security model is replaced by a rigorous approach that focuses on authenticating and authorizing every access attempt, regardless of whether it originates within or outside the network. By strictly controlling access and adhering to a *"need-only"* basis, ZT

The work reported in this paper has been supported by the National Science Foundation under grants 2112471 and 2229876.
Authors' Contact Information: Charalampos Katsis, Department of Computer Science, Purdue University, West Lafayette, Indiana, United States; e-mail: ckatsis@purdue.edu; Elisa Bertino, Department of Computer Science, Purdue University, West Lafayette, Indiana, United States; e-mail: bertino@purdue.edu.

ACM 2471-2566/2025/02-ART23
https://doi.org/10.1145/3712262

ensures that only authorized parties can access critical resources and services in the least privileged manner. Such fine-grained control mitigates the risks of unauthorized access and curtails lateral movement by attackers. It reduces the impact of compromised entities, as attackers would have limited access to sensitive resources even if the network perimeter is breached.

Problem and Scope. Enforcing strict network access control poses a significant problem for several reasons. First, achieving the least privileged network access requires a comprehensive understanding of the communication requirements of every network component, including applications, **Internet of Things (IoT)** devices, and services [10, 24, 25, 36, 46]. Second, once these requirements are identified, network administrators are tasked with manually generating appropriate flow rules to allow authorized traffic within the network. This process is highly susceptible to errors, and even the slightest oversight could lead to a chain of failures [25]. Lastly, there is often no understanding of the benign behavior of applications that utilize those permissions during data transmission, such as the typical structure of exchanged packets and data exchange patterns. *The goal of this work is thus to design an end-to-end automated framework for the learning, generation, deployment, and monitoring of ZT policies in network systems.*

We cast our design in the context of **Software-Defined Networks (SDNs)** [26] as it is an effective framework to facilitate efficient solutions based on ZT principles. The reason is that an SDN employs a centralized controller in the control plane to manage communication requests from data plane hosts and services. This centralized controller has the authority to make decisions for handling these communications, such as granting or denying access based on the established ZT policies or how the data should be routed in the data plane. Using this centralized control, the SDN enables a granular and adaptive approach to implementing ZT, allowing for enhanced security and fine-grained control over network communications.

Challenges. The design of our framework requires addressing the following challenges: **(C1)** The communication requirements of the network components are often not provided. Relevant information includes network-related intricacies, such as the allowed communication protocol stacks (e.g., `ETHERNET_IP_TCP`). **(C2)** The operation of the network components during benign conditions is not known in advance, making it impractical to effectively identify anomalous behaviors abusing allowed flows. That is, **C2(a)** the typical format of the messages and **C2(b)** the typical behavior of a particular component during data transmission are not often known. **(C3)** The correct access control rules allowing the network components to complete their missions successfully are often unknown (e.g., allow bidirectional **Transmission Control Protocol (TCP)** and **User Datagram Protocol (UDP)** communications at port 5555).

Our Approach. To address those challenges, we propose *ZT-SDN*, a ZT framework that learns the benign communication patterns and behaviors of communicating entities (i.e., users, applications, services) from the underlying network and automatically generates fine-grained access control rules using unsupervised **machine learning (ML)** techniques while also understanding how those rules should be used under benign conditions.

ZT-SDN collects and analyzes data on communications and patterns of every communicating entity. The analysis is carried out by identifying which entities use the network and how. This is achieved by collecting network traffic and network-related actions performed by applications on hosts. ZT-SDN then constructs a *communication requirements graph* in which the nodes represent the communicating entities and the edges represent the different network "transactions" carried out. The network transactions are represented by the protocol stack of the exchanged network packets. Each transaction direction can be thought of as network access (addressing **C1**). Therefore, the key idea is to extract the patterns from these accesses. We extract the patterns of the network transactions in two forms: (1) the structure of packets involved in a transaction

(e.g., protocol stacks used, port numbers) and (2) the characteristic behavior of data transmission. Those patterns are obtained using ML models and trained using datasets specific to each entity performing transactions. For extracting the structure of the packet, we use unsupervised **artificial neural networks (ANNs)** [47]. This allows ZT-SDN to determine the access patterns of benign data originating from a particular source in the graph, allowing the identification of anomalous accesses that deviate from the learned distribution (addressing **C2(a)**). To extract the behavior of data transmission, ZT-SDN observes the pattern of data transmissions on ongoing connections using a novel time series approach. The idea is to measure how far (or close) the observed patterns are to the learned patterns (addressing **C2(b)**). Those ML approaches allow for identifying deviant behavior and pinpointing the specific entity causing the issue. Deviant behavior may occur due to attacks or abnormal yet benign behavior. A typical example of the latter case is two applications with different transmission patterns, yet both of them are benign applications. We demonstrate that our approach can effectively identify the abnormality in both attack and abnormal benign patterns while exhibiting a reasonable tolerance against changing network conditions, including reduced bandwidth, link delays, and jitter.

The quality of the extracted patterns is intricately linked to the duration of the training process. However, given the diverse complexities of different applications, determining the optimal amount of data required for each can be challenging. To address this issue, ZT-SDN incorporates user-provided training heuristics as part of the solution. Such heuristics include the minimum training time and the desired performance on unseen benign training data. Then, ZT-SDN employs an iterative learning approach, continuously updating the models until the performance requirements are achieved. The training is then terminated or resumed until those requirements are met.

Finally, ZT-SDN analyzes network patterns and generates *correct* flow rules, which are installed in the data plane devices (i.e., network switches), allowing the permitted traffic to pass while restricting transactions not encountered during training. We introduce an innovative approach wherein ZT-SDN autonomously infers the appropriate protocol stack features (i.e., predicates) for rule construction and their respective values. This process involves measuring the correlations between protocol header–value pairs through association rule-mining techniques. Subsequently, ZT-SDN identifies the most extensive set of feature–value pairs that can collectively constitute the rule with the highest association measure (e.g., 100%). As a result, the constructed rules are correct, as each header–value pair strongly associates with the rest of the pairs in the dataset (addressing **C3**). Our framework not only generates the rules but also finds strong associations between them. The core concept revolves around identifying which of the generated rules apply to the traffic transmitted within a specific time window and how often. For example, as the TCP is a bidirectional protocol, the forward and backward direction rules will be automatically classified as strongly associated, thus capturing the intricacies of network protocols. ZT-SDN leverages this knowledge by deploying strongly associated rules together in the data plane. Our results show that such an approach reduces the number of interaction roundtrips between the control and data planes, thus improving efficiency.

The ZT-SDN training process is designed to be performed directly from the underlying network, assuming that all collected data are benign. However, this assumption may be impractical in many network scenarios in which adversarial absence cannot be guaranteed. To address this concern, we have developed *ZT-Gym*, an alternative approach to facilitating data generation and model training offline before integration into the SDN infrastructure. ZT-Gym operates as a Linux-based controlled environment in which relevant applications or services are deployed and executed, generating datasets for offline model training.

Novelty. To our knowledge, ZT-SDN is the first end-to-end ZT pipeline for automated learning, enforcement, and monitoring of access control policies in the context of SDNs.

Contributions. The article makes the following contributions:

- ZT-SDN, a novel end-to-end ZT architecture for SDN networks.
- Automated learning processes supported by datasets collected from the underlying network infrastructure without requiring prior knowledge about the communication requirements of applications or services.
- ZT-Gym, a virtualized environment for the deployment of software applications and offline dataset generation and model training.
- Techniques for profiling different aspects of communications executed by different applications using unsupervised ML models and making access control decisions.
- A novel technique for automatic flow rule generation and identification of strong rule associations.

2 Background

In an SDN, the network policies are installed and enforced in the network in the form of *flow rules*, in which each rule consists of a set of matching predicates (e.g., source/destination addresses, protocol, port numbers) and corresponding actions (e.g., forward, drop, modify) to be performed on matching packets. The flow rules are communicated from the network controller to the forwarding devices (i.e., switches) via the *southbound channel*, which uses a communication protocol that those devices support (e.g., OpenFlow [31] and ForCES [50]). Once a switch receives the rules, it installs them in its *flow table*. Each rule within a switch is assigned a unique identifier set by the network controller, known as a *cookie value* according to the OpenFlow specification. This unique identifier, set by the network controller, facilitates actions ordered by the controller, such as rule updates or revocations.

Typically, the forwarding rules are installed according to a *reactive forwarding* approach. In this approach, the controller installs a rule on a switch as a result of a packet receipt at an input port of the switch. The switch first checks its flow table to see whether there is any matching policy for handling the packet. The rules in the switch are matched in descending order of *priority*. The rule with the lowest priority is typically installed by the network controller (ONOS) and instructs the switch to forward the packet to the controller encapsulated in an `OFPT_PACKET_IN` message [2]. We define this as the *default rule*. This message includes the default rule's cookie (responsible for triggering the `PACKET_IN`) and the switch's identifier, providing essential information to the controller. The controller checks the request and makes a decision. If the packet should be forwarded to its destination, the controller replies with an `OFPT_PACKET_OUT` message [2] to the switch indicating how the packet should be dispatched (e.g., forward packet to output port 2). Typically, after the latter message, the controller sends to the switch an `OFPT_FLOW_MOD` message [2], which includes a rule that the switch needs to install in its forwarding table. The controller may set a rule timer in which the switch revokes the rule automatically after a period of inactivity. The controller may also set the *hard* and *idle expiration timers*, which instruct the switch to revoke the rule after the specified time. The hard timer revokes the rule after a specified number of seconds, whereas the idle timer revokes the rule only if there is no packet matching the rule for the specified number of seconds. As long as a rule has not expired, future packets matching the rule are directly handled as per the controller-specified treatment. Thus, the switch does not need to consolidate with the controller again, which saves a significant amount of time. The controller can also install forwarding rules *proactively*. Such an approach reduces the performance overhead imposed when the switch has to hold the packet transmission until a decision is received from the control plane [22]. However, one must know the policies that must be installed in advance, which may not be the case.

The forwarding decisions in the control plane are handled by *network applications*. The controller exposes control plane functionality to network applications via its *northbound channel*.

This functionality is typically exposed as **Application Programming Interfaces (APIs)** or REST APIs. An application registers to listen to specific events (e.g., `PACKET_IN`) and may provide the logic that handles those events.

3 Related Work

Network access control. Several approaches have been proposed for enforcing access control in organizational networks [6, 7, 10, 13, 25, 35, 48]. Nayak et al. [35] proposed Resonance, a framework for the specification and enforcement of access control policies. Katsis et al. [25] proposed NEUTRON, a graph-based framework for defining and enforcing least privilege access control policies. Csikor et al. [13] proposed an approach to implementing ZT on **domain name system (DNS)** infrastructure, thereby controlling client domain name resolutions based on their identity and privileges. Anjum et al. [6] proposed a framework supporting the definition of least privilege access control policies for SDNs. Vanickis et al. [48] proposed a high-level framework, FURZE, for ZT networking. The framework is composed of multiple components involving policy authoring using FURZE language for generic AC policies and firewall rules. All of those approaches have many limitations: (1) they assume that the network-wide security policies are provided by some administrator; (2) they assume that policies are correct and detailed, thus, the generated rules allow "need only"–based communication; (3) they do not capture behavioral aspects of the communications, such as the message or data transmission patterns that are essential to understanding how the permissions are used in benign conditions; and (4) the predicates used in the generated rules are limited to administrator-specified fields (e.g., source and destination addresses and ports). All of these limitations are addressed by ZT-SDN in an automatic manner.

Rule mining. Golnabi et al. [19] proposed an approach to mine-generalized firewall policy rules from network traffic logs based on rule frequency analysis. Apiletti et al. [8] proposed the NetMine framework to extract generalized rules from network traffic data using association rule mining. Those approaches have the following limitations: (1) the rule extraction is limited to user-specified predicates or user input; (2) the generalization process is not designed with least-privilege network access in mind, which results in rules allowing a potentially wide range of hosts to access network resources. ZT-SDN addresses those limitations. It analyzes the packets transmitted between different hosts separately. Then, it uses a novel rule-generation approach that automatically extracts the predicates and uses them for constructing rules with high confidence and support. Finally, ZT-SDN generates rules specific to the communicating entities, restricting access to resources not specified in the **Communication Requirements (CR)** graph.

Anomaly detection in SDN flows. El Sayed et al. [15] proposed a supervised learning method based on Long Short-Term Memory and AE to detect **distributed denial of service (DDoS)** attacks. Peng et al. [39] proposed another supervised approach based on the **K-Nearest Neighbor (KNN)** algorithm for detecting DDoS attacks. Da Silva et al. [14] developed the ATLANTIC framework, which identifies flow anomalies based on entropy analysis and a supervised algorithm based on **support vector machines (SVMs)**. Garg et al. [17] proposed a supervised deep learning–based anomaly detection scheme for suspicious flow detection in the context of social multimedia. Additionally, Nanda et al. [34] evaluated various supervised ML algorithms trained on historical network attack data to predict potential malicious flows. Unsupervised approaches have also been proposed to detect anomalies based on techniques such as entropy analysis, clustering, and isolation forest [3, 9, 42].

In contrast, our ZT-SDN framework offers a novel approach, fundamentally different from prior works. Rather than focusing solely on detecting ongoing attacks, we aim to prevent abuse of network permissions by entities by understanding how those permissions are used in benign

conditions. Our approach distinguishes itself in three key aspects: (1) it operates entirely in an unsupervised manner; (2) it tailors the training to the specific entities using the network permissions, thereby understanding how the communicating entities typically use the network permissions; and (3) it detects anomalies in granted flow permissions using time series modeling to assess pattern similarities between observed and learned patterns (e.g., sending too fast/slow or too much/few data).

Relation to NIDSs. While ZT-SDN and **Network Intrusion Detection Systems (NIDSs)** share complementary goals in network security, critical distinctions set them apart. NIDS technologies function at the data plane, continuously monitoring traffic streams for anomalies based on diverse indicators, such as connection resets, SYN attempts, and traffic patterns [21, 28, 33]. This monitoring allows a NIDS to classify traffic and detect attack signatures accurately. However, a NIDS does not typically perform access control enforcement; it is typically deployed at specific network ingress or egress points and thus cannot comprehensively mediate all traffic between internal network entities. Deploying NIDSs to every access point across all subnets would lead to prohibitive costs and complex management challenges. Additionally, relocating NIDS functionality to the control plane is impractical owing to the overwhelming volume of data plane traffic, which would exhaust the control plane's resources.

The NIDS provides alerts on detected anomalies but does not handle access control decisions, meaning it cannot assess whether a communication should be permitted or denied. Furthermore, the core objective of the NIDS is to detect attacks rather than manage deviations in network permissions assigned to specific components, making it unable to distinguish subtle variations between legitimate traffic flows. In contrast, ZT-SDN operates within the control plane, enabling it to dynamically enforce network-wide access control policies. ZT-SDN actively decides whether packets should be allowed or blocked, and it can analyze flow statistics in real time, aligning with OpenFlow specifications, to detect deviations in permitted network behavior. Thus, while ZT-SDN and the NIDS both contribute to network security, they cannot be directly compared or replace one another.

4 Design of ZT-SDN

In this section, we first present an overview of the ZT-SDN architecture, followed by an outline of the threat model and assumptions.

4.1 Architecture Overview

ZT-SDN has three modules (Figure 1): the **host module (HM)**, the **controller-specific module (CSM)**, and the ML module. The HM provides information about which applications on the hosts require network access (addressing **C1**). The CSM is a network application that consumes the northbound interface exposed by the network controller (i.e., **the Open Network Operating System (ONOS)**). Thus, the execution of the network application has to be adjusted to the underlying controller. This module performs a different set of operations depending on its mode:

Training mode. It identifies the communicating entities by analyzing network traffic from the data plane. It also leverages information from the HM instances to map specific traffic streams in the data plane to the corresponding applications running on the hosts. This mapping facilitates the extraction of a *communication requirements (CR)* graph, representing different sets of communications (edges) between the identified entities (nodes). For each edge in the graph, the CSM generates datasets used by the ML module for training the models and extracting the communication patterns (addressing **C1, C2**). Once the models have been trained, the module transitions into enforcement mode.

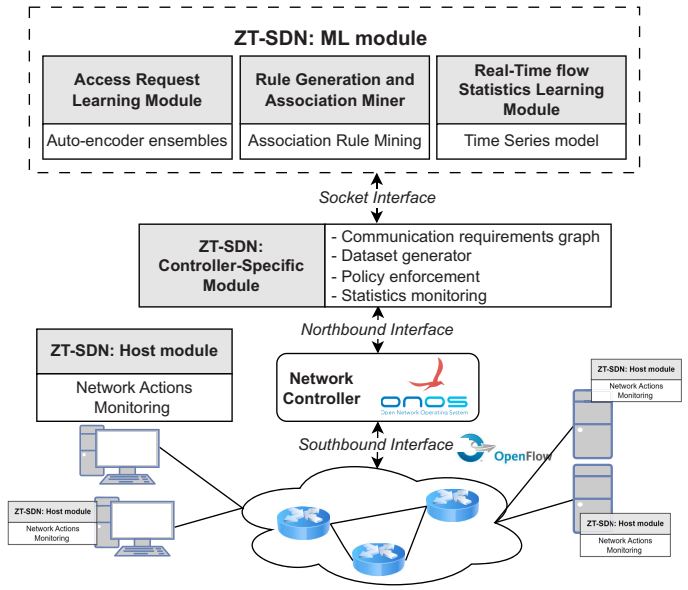


Fig. 1. The ZT-SDN architecture.

Enforcement mode. In this mode, the CSM uses the ML module to make network access control decisions. It also constructs the rules generated by the ML module and installs them at the network's appropriate switches (i.e., **policy enforcement points – PEPs**). It retrieves flow statistics associated with the installed flow rules in real time and provides them to the ML module.

The ML module is a controller-independent module, thus allowing interoperability across different SDN controllers. Like the CSM module, it operates in two modes:

Training mode. During this mode, the ML module processes the data plane traffic datasets. Utilizing unsupervised learning techniques, it extracts the benign communication patterns modeled by the CR graph and derives corresponding flow rules (addressing **C1, C2, C3**).

Enforcement mode. In this mode, the trained models authorize network access requests from hosts in the data plane based on temporal factors and packet headers. It also continuously analyzes flow statistics of ongoing communications to identify potential deviant behaviors.

4.2 Threat Model and Assumptions

ZT-SDN operates under the assumption that no prior knowledge is available regarding the applications running on the hosts in a network. Additionally, we assume that there is an authentication process for the host machines before the training process commences. This is essential, especially in cases in which the entity network identifiers (e.g., **Internet protocol (IP)** or **media access control (MAC)**) change over time. Therefore, endpoint identity is used for training and access request authorization.

Throughout the ML training phase, it is crucial that the host applications running are not compromised. However, once the model is trained and the ML and CSM modules switch to the enforcement mode, an adversary may compromise host applications or services running inside or outside the network. Finally, we assume that the control plane operations (i.e., network controller), the southbound channel and the data plane forwarding devices (i.e., the PEPs) are never compromised at any point in time.

Nevertheless, the assumption that the host applications are not compromised during the training phase may be overly stringent for practical adoption in some organizations, particularly in scenarios in which network administrators cannot guarantee the absence of abnormal network activity, even during the training phase. To address this concern, we developed an *offline approach* to dataset generation and model training. In this approach, applications are deployed and executed within a controlled "gym environment," referred to as *ZT-Gym* – a virtualized setting in which the application(s) under scrutiny run. Network traces collected from *ZT-Gym* are then used for training the models for deployment in the network. We provide technical details of this alternative approach in Section 11. The approach discussed in the rest of the section performs the training phase directly from the network.

5 Host Module (HM)

The HM is deployed on endpoint systems as a lightweight Python program designed to continuously interface with the underlying **operating system (OS)** to detect network-related events generated by system processes. These events include port bindings and data transmissions. To accomplish this, it uses the Linux utility `ss-plan` to identify ports bound by applications [5]. The collected data includes the timestamp of event detection, applications binding to OS ports, identification of applications transmitting data, the transport-layer protocol in use, and destination details such as IP addresses and ports. This information is subsequently transmitted to the CSM, enabling the correlation of network flows with their respective host applications.

Support for Embedded Systems. The HM's installation is not required on embedded systems, including the IoT and operational technology control devices. Given that these devices execute highly specific tasks, they can be treated as standalone applications.

6 The Controller-Specific Module (CSM)

The CSM builds upon the ONOS's reactive forwarding application, known as Fwd, to manage the `PACKET_IN`, `PACKET_OUT`, and `FLOW_MOD` operations, which facilitate effective communication between entities in the data plane. The initial step involves the CSM registering with the network controller via the northbound interface and requesting to handle any `PACKET_IN` messages from the data plane.

Training mode. When a packet is sent from a host in the data plane, the switch sends it to the CSM encapsulated in the `PACKET_IN` message. This happens when there is no existing flow rule permitting communication, essentially indicating a network access or communication request from a source to a destination. Subsequently, the CSM extracts the source and destination addresses from the packet, along with the packet's protocol stack (e.g., `ETHERNET_ARP` or `802.11_IP_TCP`).

The CSM identifies the communicating hosts using the extracted information and determines the applications involved through HM updates. It then utilizes this knowledge to create the CR graph. In such a graph, each node represents an application running on a host, encompassing both client-side applications and network services. The directed edges between the nodes represent the identified protocol stack used for communication. Essentially, each edge in the graph represents a necessary network access, which is defined by the source and sink nodes and the protocol stack of the transmitted packets over that edge. The CR graph represents the essential flows required for applications to perform their intended functions successfully. Additionally, it provides insights into the endpoints, such as IP addresses and domain names, with which applications typically communicate.

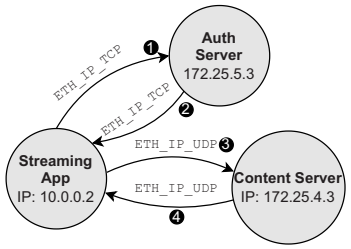


Fig. 2. An example of a simple communication requirements graph.

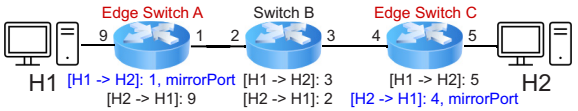


Fig. 3. Rule deployment for traffic collection and forwarding. The notation is [matching packet headers]:[List of forwarding interfaces].

Figure 2 shows a simple example of a CR graph. The example illustrates a streaming application that uses two protocol stacks: ETHERNET_IP_TCP (flows (1)–(2)) for the authentication procedure and ETHERNET_IP_UDP (flows (3)–(4)) for content consumption.

For every discovered edge in the CR graph, the CSM generates two types of datasets: (1) packet datasets and (2) flow statistics datasets. The packet datasets capture all transmitted packets associated with a specific edge in the graph. For instance, if an application communicates with TCP and UDP protocols, the CSM will generate two individual datasets for the application: one for the TCP packets and one for the UDP packets. The CSM currently supports various protocols, such as **Address Resolution Protocol (ARP)**, IP, **Internet Group Management Protocol (IGMP)**, **Internet Control Message Protocol (ICMP)**, TCP, and UDP. Each entry in the dataset contains temporal information (i.e., day, time) and the parsed packet header, which includes protocol-level details such as addresses, **virtual local area network (VLAN)** ID, frame types, ports, and flags.

The CSM then sends a PACKET_OUT command to forward the packet to its intended destination. The CSM installs a custom rule using the source and destination addresses, ports, and the transport layer protocol. This rule ensures that subsequent packets related to the same flow can be efficiently forwarded to their destinations. The action set of the rule is configured to forward the packet through a designated egress port determined by the ONOS's topology service [38] at the time of rule installation. For the *edge* switch[1], an additional action is configured in the flow rule. This additional action instructs the edge switch to send a copy of matching packets to a *mirroring* port[2] (Figure 3). The mirroring port of each edge switch is connected to an end system that passively collects data, such as the controller or organization-owned hosts[3]. While the CSM can collect traffic data passively at the controller, this may consume significant bandwidth and processing power, particularly with numerous hosts connected to an edge switch. This can slow down control and data plane operations. Therefore, a more scalable alternative is for the CSM to obtain traffic data separately from the mirroring systems.

[1]The edge switch is the switch where a host attaches to the network.
[2]Port mirroring is typically supported by hardware for efficiency [11].
[3]There are several efficient approaches for packet capturing at least 100 Gbps, such as [16]. This choice does not impact ZT-SDN.

Upon installing the corresponding rule in a switch's flow table, the CSM utilizes the ONOS flow statistics service [26] to generate datasets for different CR graph edges. For each flow direction, the CSM obtains flow statistics for the edge switch connected to the source host. For instance, in Figure 3, the flow H1→H2 is queried at switch A, while the flow H2→H1 is queried at switch C.

The flow statistics that can be collected by OpenFlow switches are predefined by the OpenFlow specification [2]. The available statistics for each flow include the total number of packets and bytes transmitted since the installation of the flow rule until the time of the query. Thus, ZT-SDN has access to those statistics only. The queries are performed by the controller at predetermined intervals of f seconds. At the time of the query t, the number of transmitted packets and bytes are computed as follows:

$$\#packets_t = \sum_{rule \in edge.rules} rule.packets_t \tag{1}$$

and

$$\#bytes_t = \sum_{rule \in edge.rules} rule.bytes_t. \tag{2}$$

In essence, the number of packets (or bytes) at t is computed as the sum of packets (or bytes) for each installed flow rule associated with the specific edge in the CR graph. Each entry in the dataset comprises the query time (t) along with the total number of packets ($\#packets$) and bytes ($\#bytes$) reported during that query.

ZT-SDN performs data collection operations at edge switches instead of core switches because (1) edge switches mediate the communication even in multi-path packet propagation, and (2) edge switches typically handle less traffic load than core switches, allowing for more scalable data collection.

Enforcement mode. The CSM parses the `PACKET_IN` message (similar to the training mode) and performs a query on the CR graph to determine whether the specific communication has been previously modeled during the training phase. If it finds a match in the graph, the packet is then forwarded to the **access request learning (ARL)** module for verification against the learned distribution of packet headers. Upon successful verification, the CSM receives the flow rules generated by the **rule generator and association miner (RGAM)** module and proactively deploys them across all switches between the source and the destination. The path for rule deployment is determined by the ONOS at the time of rule deployment. Similar to the training mode, the action set for the rules is set to "forward" and the egress ports are specified based on the path determined by the ONOS. However, if the communication has not been modeled during training in the CR graph or if the packet does not belong to the learned distribution (as determined by ARL), the communication request is denied. ZT-SDN allows network administrators to set hard and idle flow timers for the deployed rules. By default, it sets an idle timer at 10 seconds so that inactive rules are retracted from the data plane.

The ZT-SDN system allows network administrators to dictate whether copies of subsequent packets, for either all or specific edges in the CR graph, should be forwarded to the controller during enforcement mode; this can even be set for a specific time duration. This decision, however, involves balancing security and network performance. Forwarding packet copies to the controller enables the ARL module to conduct more thorough security checks, thereby enhancing network security. Conversely, this method increases bandwidth usage, negatively impacting network performance. Therefore, administrators need to consider these aspects and decide what best aligns with their specific security and performance needs.

Following the deployment of the rules, the CSM continues to query the statistics of the installed flow rules in the edge switches (similar to the training period), allowing for real-time flow

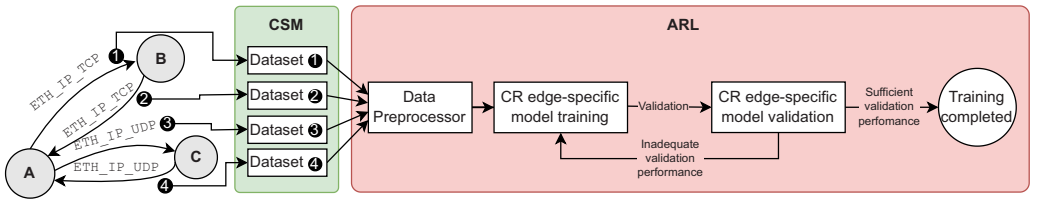


Fig. 4. The model training procedure for each CR graph edge.

monitoring. The collected statistical information is passed to the **real time flow statistics learning (RTFSL)** module, which actively checks for any deviations in the usage of allowed network flows. If any deviation is detected, the CSM ends the communication between the endpoints by removing the corresponding flow rules from the data plane, effectively cutting off the communication.

Network Overheads. The processing of PACKET_IN requests in ZT-SDN increases due to the CSM processing and the access control decision made by the ARL module. However, as demonstrated in Section 12.5, this increase in processing is offset by the proactive rule deployment approach. The RTFSL module does not cause network overhead as the ONOS controller obtains flow statistics from all network switches at a predetermined frequency. Thus, it utilizes the statistics that have already been collected. Lastly, rule generation is done once during training; thus, it does not add overhead during enforcement.

7 The Access Request Learning Module (ARL)

The module's objective is to determine whether to grant or deny a communication request originating from an entity in the data plane by evaluating the packet contained in a PACKET_IN request. To achieve this, the module must learn the allowed values for the different protocol stack headers typically used for the specific edge in the CR graph, such as the permissible port numbers, time-to-live values, typical header and packet lengths.

Training. PACKET_IN requests may contain a packet that can be exchanged at any point in time between entities, as connections may be interrupted and later resumed from where they left off. Therefore, training solely on the initial packets of an edge is insufficient; the module must be able to recognize any benign exchanged packet. The ARL module receives traffic datasets generated for each application from the CSM, with each dataset corresponding to a different protocol stack. Figure 4 shows the ARL training process. Each sample in these datasets contains the day and time of packet receipt along with the protocol header values.

To prepare the data, we first apply protocol-specific processing, which involves feature transformations and deletions. For instance, to preserve the semantic meaning of the TCP flags, we one-hot encode the TCP flags header to differentiate the set flags for each packet. Protocol features with no meaningful values for the access control learning process are removed, such as acknowledgment values and checksums. Then, standard scaling is applied, which removes the mean and scales the data to unit variance. We then train a model composed of **autoencoder (AE)** ensembles for each CR edge to capture the distribution of packet headers in that edge. An AE is a neural network architecture used for unsupervised learning, primarily designed to learn efficient representations of input data by reconstructing the original input from a compressed intermediate representation. The goal of an AE is to minimize the reconstruction error–the difference between the original input and the decoder output–thus learning essential features and patterns of the input data in the latent space.

We specifically use the KitNet model [33], which is a lightweight, highly efficient **central processing unit (CPU)**–based model built on AE ensembles. KitNet uses a feature mapper to create smaller clusters of features, with each cluster fed to a separate AE in the ensemble. This allows the AE ensemble to capture and learn the distinguishing aspects of normal network behavior, enabling the model to effectively identify deviations and anomalies.

KitNet operates in two modes: training and execution. Initially, it requires the specification of the number of training examples. Once this number of samples has been observed, KitNet transitions to the execution mode. In this mode, the model calculates the reconstruction error for all subsequent samples. However, this approach poses a significant challenge, as determining the precise number of training samples needed beforehand can be difficult. First, different applications vary in complexity, necessitating a unique number of training instances for each application-specific model. Second, the model must capture the temporal factors associated with the access request, such as the time of the request. To address these issues, we have modified the algorithm's design by allowing network administrators to provide heuristics on the desirable model performance on both seen and unseen training data. We perform the following modifications:

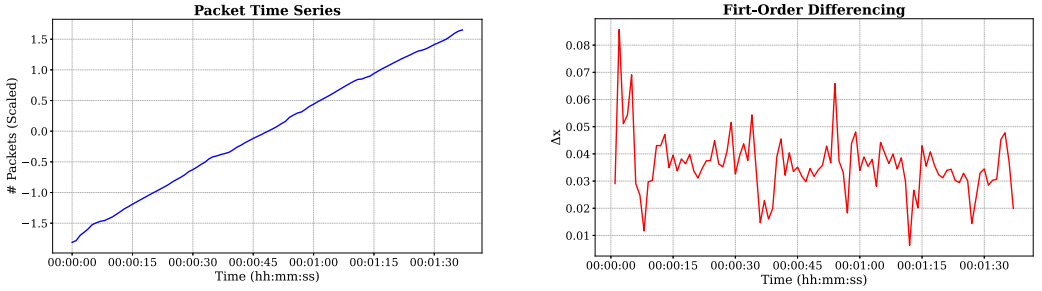
(1) Introduction of the minimum training time. This refers to the minimum duration during which an application is expected to demonstrate its typical behavior associated with temporal factors.
(2) Introduction of the minimum number of training samples. We establish a minimum threshold for the number of training samples required to adequately train the model (e.g., 20K samples).
(3) Incorporation of a validation period. A designated time span is introduced to assess the model's performance on unseen benign data.
(4) Introduction of a maximum reconstruction error threshold. During the validation period, we define a maximum reconstruction error threshold. If the average reconstruction error during the validation period falls below this threshold, indicating satisfactory performance, the model proceeds to the execution mode. Otherwise, it resumes training to improve its accuracy.

Therefore, the model training becomes an iterative process in which the model is updated until the user-provided performance requirements are met. The model observes data for a minimum duration and a minimum number of training samples. Then, its performance during the validation period determines the transition to the execution mode.

Enforcement. The packet is extracted from the `PACKET_IN` message and the endpoint addresses are used to identify the corresponding edge in the CR graph. The packet is then preprocessed and passed through the scaler, followed by the inference of the KitNet model trained on that edge, which either allows or denies the communication.

8 The Real Time Flow Statistics Learning Module (RTFSL)

When an access request is granted based on the CR graph and the ARL module, a flow rule or policy is installed, allowing the traffic to proceed to its intended destination. However, from a ZT perspective, we want to ensure that the granted network permissions are used as per the learned transmission patterns. For instance, a resource that typically transmits very little data in an active flow starts transmitting rapidly or at the same rate with very different payload sizes. Therefore, the RTFSL module addresses two critical requirements: (1) learning the normal flow usage permitted by the access rules for the modeled entities in the CR graph and (2) detecting deviations from the learned communication patterns and terminating the flows.



(a) Packet Time Series ($\{x_t\}_{t=1}^{n}$)

(b) First-Order Differencing Time Series (Δx_t)

Fig. 5. Transformation of the packet time series (a) to the first-order difference representation (b).

ALGORITHM 1: Traffic Pattern Similarity Checker

Require: $\{\Delta x_t\}_{t=2}^{n}$, New Observations
1: $new\_obs \leftarrow$ first_order_diff(New Observations)
2: $min\_dist \leftarrow \infty$
3: **for** i in range(size($\{\Delta x_t\}_{t=2}^{n}$) - size(new_obs) + 1) **do**
4: $segment \leftarrow \{\Delta x_t\}_{t=2}^{n}[i : i + size(new\_obs)]$
5: $dist \leftarrow \text{DTW}(new\_obs, segment)$
6: **if** $dist < min\_dist$ **then**
7: $min\_dist \leftarrow dist$
8: $min\_match\_segment \leftarrow segment$
9: **end if**
10: **end for**

Training. The RTFSL module uses the flow statistics datasets generated by the CSM for training. The collected statistics are limited to the ones supported by the OpenFlow specification [2], ensuring the compatibility of our approach with the specification and, by extension, the vendor implementations. The training examples are essentially measurements of the transmitted packets and bytes transmitted over time. The data are sampled from a network switch at a predefined frequency, f, and the total numbers of packets and bytes are computed as per Equations (1) and (2). Thus, due to the data's semantic meaning and structure, we model it as a time series to capture the inherent dependencies and patterns present in time-dependent data.

We first scale the training data of the CR edge using standard scaling, which removes the mean and scales to unit variance. We then remove the trend to reveal the data transmission patterns over the sampling time, as shown in Figure 5. Figure 5(a) shows the cumulative packet transmission behavior of a real client machine transmitting to an HTTPS service from the MAWI 2024 traffic dataset [49]. To remove the trend, we apply first-order differencing to the data as follows: Given a time series $\{x_t\}_{t=1}^{n}$, the first-order differencing is formally defined as

$$\Delta x_t = x_t - x_{t-1}, \quad \text{for } t = 2, 3, \ldots, n, \tag{3}$$

where Δx_t represents the first-order difference of the series at time t. Figure 5(b) illustrates Δx_t, which clearly shows the behavioral patterns between consecutive observation samples.

The idea is to collect long enough Δx_t time series from the CR's edge such that future benign observations from the same edge can be matched to the Δx_t model with low error. We develop an approach to detect behavioral deviations in ongoing traffic flows (Algorithm 1) from the series $\{\Delta x_t\}_{t=2}^{n}$. We query the edge's traffic flows with the same sampling rate, f, for some

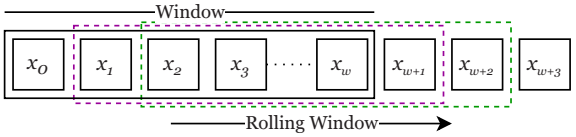


Fig. 6. RTFSL rolling window.

pre-determined time duration, d. From the observation, we compute the first-order difference (line 1) of the measurements. We perform **dynamic time warping (DTW)** [41], which is a distance measure that quantifies the similarity between two-time series (line 5). DTW considers the possible variations in the alignment and scaling of the time series, making it suitable for matching time series with different lengths and shapes. We use the Euclidean distance as the distance metric between the two time series. Our experiments with various time series shapes and sizes have shown that Euclidean distance is a good quantification metric for measuring similarity between them. Finally, we set a maximum distance threshold in which patterns that exceed the threshold are added to the $\{\Delta x_t\}_{t=2}^{n}$ series. We use the FastDTW implementation of DTW [29, 41], which has an $O(n)$ time and space complexity, where n is the length of the time series.

We use four heuristics to stop the learning process:

(1) We use a predefined minimum data duration in which the model should be able to capture all the potential traffic patterns.
(2) We use a minimum number of training samples in the $\{\Delta x_t\}_{t=2}^{n}$.
(3) We define a predefined validation period and a maximum Euclidean distance threshold to ensure that the model works well on unseen benign data.

Enforcement. During model enforcement, the new observations are scaled and compared with the learned $\{\Delta x_t\}_{t=2}^{n}$ using Algorithm 1. We then define an anomaly threshold, which, if it exceeds the pattern, is recognized as anomalous.

We define a window size, w, which denotes the number of consecutive flow statistics samples, as shown in Figure 6. The window of the samples is run through Algorithm 1 to evaluate the observed behavior against the learned behavior. The process continues on a rolling basis as long as the flow is active.

Model Effectiveness. From our evaluation results, we observe several key strengths of the proposed model. First, the fine-grained analysis provided by our novel CR graph enables the extraction of transmission behaviors along the graph's edges. Second, this level of analysis allows us to identify not only deviations in attack patterns, such as flooding attacks, but also deviations in between other benign traffic patterns, enabling the model to distinguish between different uses of the granted network permissions. Finally, we found that the approach demonstrates a reasonable resilience to statistical fluctuations caused by changing network conditions, including delays and jitter. Notably, the model remains effective even when trained under conditions different from those present during enforcement, showcasing its adaptability across diverse operational scenarios.

9 The Rule Generator and Association Miner Module (RGAM)

This module has a twofold objective. First, it generates network access control rules specific to each edge in the CR graph based on the observed packet traffic during training. Our approach automatically infers which protocol header attributes (i.e., predicates) from the observed traffic should be used to construct those rules. For instance, the predicates could be the Ethernet type, ARP operation type, ICMP code, protocol codes, IP addressing, and transport layer ports. In the

case of the example in Figure 2, it will identify which predicates are needed for permitting each of the four flows. The generated rules will eventually be installed in the network switches if the appropriate access request is made.

This module, therefore, generates only *positive rules*, meaning that the generated rules only allow the appropriate traffic to pass. Traffic that does not match any generated rules will be matched to the default rule and, therefore, sent to the controller. The ARL makes the access control decision and denies the traffic if it does not belong to the learned packet distribution.

Second, the RGAM utilizes the generated rules to identify strong associations among them. The underlying concept is that when a rule is scheduled for deployment based on an access request, it is efficient to deploy all strongly associated rules proactively. Such a strategy helps minimize the exchange of `OFPT_PACKET_IN`, `OFPT_PACKET_OUT`, and `OFPT_FLOW_MOD` messages between the control and network switches, thereby reducing the overhead caused by these control plane operations. Moreover, identifying rule associations enables ZT-SDN to learn the unique communication characteristics of different protocols. For example, in the case of the TCP, bidirectional flows are required between the endpoints, whereas in the UDP, the receiver is not obligated to respond to the sender. In the illustrated example in Figure 2, the RGAM finds strong correlations between rules of the same protocol stack (i.e., rule pairs for flows (1)–(2), and (3)–(4) are strongly associated) as well as between rules of different protocol stacks (i.e., rules for flows (1) to (4) are strongly associated). Therefore, once the streaming application makes an approved access request for the authentication server, all permitted flows are proactively deployed, saving communication overhead and allowing the application to operate successfully.

The RGAM is based on association rule mining, a technique for discovering relationships and patterns in datasets [4, 27]. It focuses on identifying frequent itemsets and extracting association rules based on co-occurrence patterns in the data. Association rules capture the dependencies and associations between different items or attributes in a dataset. These rules are typically in the form of $\{antecedents\} \Rightarrow \{consequences\}$ statements, where certain attributes in a transaction (i.e., the antecedents) imply the presence of other attributes (i.e., the consequences).

Each generated rule has metrics to measure how strong the association is. First, the *support* metric is defined as

$$P(A \cap B) = \frac{freq(A, B)}{N}, \tag{4}$$

where A and B are itemsets (i.e., sets of one or more attributes) of various lengths, and $P(A \cap B)$ is the joined probability of A and B. Thus, support measures the occurrence of a rule in a dataset. Second, the *confidence* metric is the conditional probability defined as

$$P(B|A) = \frac{support(A, B)}{support(A)}. \tag{5}$$

Thus, confidence measures how often itemset B appears with itemset A given the frequency of A.

Rule Generator. It takes as input the CR graph and an application represented as a node in the CR graph. The output is the inferred rules. A rule is defined as

$$rule = [key_1 = value_1, \ldots, key_n = value_n], \tag{6}$$

where a key is a protocol header name (e.g., *source_ip*), $value$ is the value of that header, and n is the rule's cardinality. A network packet matches a rule if all header-value pairs present in the rule are also present in the packet's header.

Algorithm 2 shows the pseudo-code of our rule generator algorithm. We limit the processing to all of the packet header features available for enforcement per the OpenFlow standard [2], such as VLAN ID, Ethernet/IP addresses, protocol flags and options, and ports. First, we fetch a subgraph

ALGORITHM 2: Rule Generator

Require: An application identifier, CR graph

```
1: all_streams ← CR.get_streams(app)
2: binary_tables ← Ø
3: for stream in all_streams do
4:     columns ← Ø
5:     for column in stream.columns do
6:         columns ← columns+ get_unique(column)
7:     end for
8:     table ← stream.no_packets × columns
9:     binary_tables.add(table)
10: end for
11: populate(binary_tables)
12: generated_rules ← Ø
13: for table in binary_tables do
14:     freq_items ← apriori(table, min_support)
15:     rules ← assoc_rules(freq_items, min_confidence)
16:     rules ← rules.antecedents ∪ rules.consequents
17:     rules ← unique(rules)
18:     generated_rules.add(max_cardinality(rules))
19: end for
```

(i.e., a part of the CR graph), which contains all nodes and edges (i.e., streams) for which the application is a source or a sink (i.e., the application and the destination services). The traffic pertaining to the edges of the CR subgraph is essentially the traffic datasets used to train the ARL module. As mentioned earlier, the ARL module determines those datasets' length dynamically until the distribution is learned. Once the dataset sufficiently captures the distribution, it is used to mine access control rules, a one-time process.

Subsequently, the algorithm prepares the binary tables, one of each stream direction (lines 3–10). The columns of a binary table are the unique header–value pairs of each column in the flow dataset. For instance, if the *source_port* attribute has three unique values in the dataset, 4455, 5588, and 6699, then that would result in three columns in the binary table: *source_port* : 4455, *source_port* : 5588, and *source_port* : 6699. The number of rows would equal the total number of packets in the stream.

Next, the algorithm populates the binary tables. For each packet (i.e., row), we set the value of a cell to 1 if the condition of the column holds for the packet; otherwise, we set it to 0. Finally, we apply the Apriori algorithm [43] (line 14) with a predetermined minimum support value (90% for the experiments). The generated frequent itemsets are then used to generate the association rules (line 15). The algorithm uses a high minimum confidence score threshold (100% for the experiments) to eliminate the less confident associations. For each generated rule, we then unify the antecedences and consequences (line 16) to create sets of header–value pairs and discard any duplicate sets (line 17). Finally, the algorithm stores the rules of maximum cardinality.

The generated rules are considered *correct* owing to the use of high support and confidence scores. However, *completeness* is not always guaranteed, as this requirement heavily relies on the intricacies of applications. For instance, consider an application that employs two communication channels with a particular service. If the second channel rarely appears in the dataset, there might be scenarios in which a rule allowing this infrequent flow is not generated. This is because communication occurs so rarely that no rule with sufficiently high confidence or support can be derived.

ALGORITHM 3: Rule Association Miner

Require: Generated rules, Application's traffic dataset

```
1:  binary_table ← 1 × rules
2:  while True do
3:      if ws = 0 then
4:          packet_time ← app_data[0].timestamp
5:          ws ← packet_time
6:      end if
7:      we ← ws + w_duration
8:      packets ← get_packets(app_data, [ws, we])
9:      populate(binary_table)
10:     index ← last_packet(packets).index
11:     if index + 1 < app_data.length then
12:         index ← index + 1
13:         ws ← app_data[index].timestamp
14:     else break
15:     end if
16: end while
17: freq_items ← apriori(binary_table, min_support)
18: associations ← assoc_rules(freq_items, min_confidence)
```

To address this issue, ZT-SDN generates such rules at access request time. This means that if a PACKET_IN request is authorized by the ARL module and the generated rules for the application have already been installed but a packet does not match any of those rules, ZT-SDN dynamically creates a proprietary rule based on the packet's source/destination address, port numbers, and protocol. Specifically, ZT-SDN maintains a record of the rules installed in the data plane for particular endpoints. If packets still arrive at the control plane despite the installed rules, it follows that the packet cannot match the installed rules. Thus, ZT-SDN will generate a dynamic rule to accommodate this rare transaction provided that the ARL module authorizes the packet. This ensures that the transmission can be accommodated even for flows that were infrequent or not explicitly extracted during the rule generation phase.

Rule Association Miner. Once the rule generator computes the rules, we find strong associations between those rules (see Algorithm 3). First, we construct a binary table (line 1) with columns representing the generated rules from Algorithm 2. Then, we compute the initial time window (lines 3–7) and get the packets within a predefined window size (line 8). The algorithm checks which rules (i.e., columns) apply to the packets in the window and sets the corresponding cells to 1; otherwise, to 0 (line 9). Then, the next time window is computed (lines 10–15). We use packet indexes to compute the time window as the application dataset may have time gaps. Thus, the algorithm saves a lot of loop iterations by skipping time windows that do not include any packets. Finally, the rule association is computed using the Apriori algorithm (lines 17 and 18) while setting the minimum support and confidence to high values (90% and 100% for the experiments, respectively).

10 Relation to the ZT Paradigm

This section explores how ZT SDN aligns with the broader principles of the ZT network paradigm.

The **US National Institute of Standards and Technology (NIST)** defines the ZT architecture in its special publication [45]. This document provides an abstract framework for ZT, along with general deployment models and scenarios.

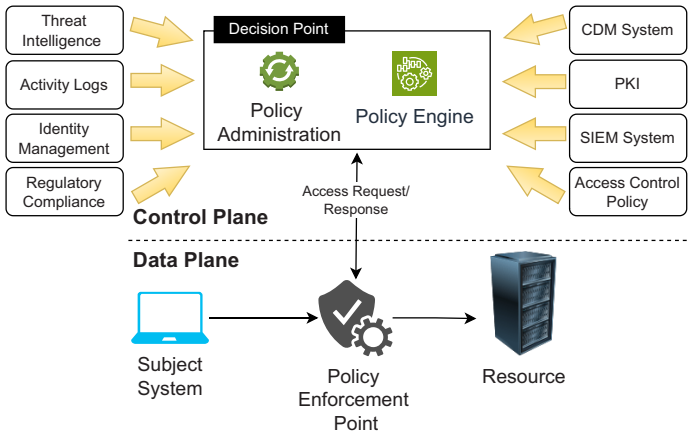


Fig. 7. Core logical components of Zero Trust architecture as defined by NIST [45].

NIST's Core ZT Components. Figure 7 shows the core logical components of the ZTA as defined by NIST. When a *subject system* (e.g., user device, IoT) requests access to a *resource*, the request is mediated through a ***Policy Enforcement Point*** **(PEP)**. The PEP forwards the request to a **Policy Decision Point (PDP)**, which comprises two primary components: the ***Policy Administration*** **(PA)** module and the ***Policy Engine*** **(PE)**.

The PA processes the request and passes it to the PE, which makes an access decision–authorizing or denying the request–based on inputs from various sources. These inputs include security policies, threat intelligence, endpoint health and patch status (monitored via **Continuous Diagnostics and Mitigation (CDM)** systems), and the identities of the involved endpoints. The PE decision-making process integrates this intelligence to determine trustworthiness.

Although the NIST framework specifies these logical components, it leaves the design of the decision-making algorithms and the detailed implementation of these modules as open research challenges. Based on the PE's decision, the PA enforces the policy by performing actions such as issuing authorization tokens, generating configurations, or deploying rules.

Mapping to the ZT-SDN Architecture. The principles of the NIST ZT framework can be directly mapped onto the ZT-SDN architecture.

In ZT-SDN, an access request is represented by the first network packet sent from a subject system (e.g., source) to a resource (e.g., destination). This packet is initially routed to an OpenFlow-enabled network switch, which acts as the PEP in the NIST model. If the switch lacks a matching policy (i.e., a rule), it forwards the packet to the control plane using a `PACKET_IN` message. In the control plane, the CSM module corresponds to the PA component. The CSM processes the `PACKET_IN` request and forwards it to the ML module, which maps to the PE. Here, the ARL module evaluates and authorizes or denies the flow, whereas the RTFSL module performs communication monitoring in alignment with NIST's ZT requirements. Finally, the *CSM* (acting as the PA) processes and deploys the rules generated by the RGAM module. These rules are then installed on the relevant data plane switches to ensure enforcement across the communication path.

11 ZT-Gym

In this section, we present our offline approach for creating application network datasets. The datasets are generated within a controlled environment devoid of any adversarial presence, ensuring the production of benign data. These generated datasets are then employed in the ZT-SDN training mode as outlined in Section 4.1.

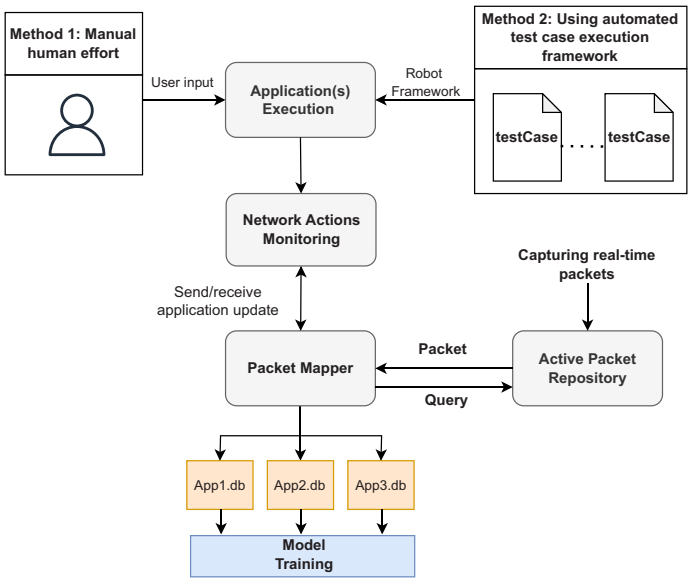


Fig. 8. The ZT-Gym environment.

11.1 The ZT-Gym Environment

We developed ZT-Gym as a Linux-based virtual machine and a dataset generation pipeline, shown in Figure 8. ZT-Gym supports the dataset generation for several applications simultaneously. The actual applications or services are installed within the ZT-Gym environment. The data (i.e., network traffic) is obtained directly from the network interface(s) to ensure that the generated data are realistic and accurate.

11.2 Technical Details

Application input methods. To generate network data, the application requires input. We support two input methods: (1) Manual human interaction—a human user interacts with the application in the virtual environment, performing the tasks it would typically do in normal conditions, and (2) an existing framework for application testing automation.

The first method is straightforward but requires human time, which is expensive. To address this, we use the Robot Framework [23, 30], an open-source automation framework that streamlines the execution flows of applications with graphical user interfaces or those run via the command line. Constructing test cases using the Robot Framework is an intuitive and straightforward process. To write a Robot Framework test case, one needs to define settings, variables, and test cases in a plain-text file with the ".robot" extension. The Robot Framework is widely adopted in corporate settings for automated testing. Companies often possess their own .robot files tailored for applications, developed as part of the software development life cycle. In this method, the test case executes the actions that the application would normally exhibit in benign conditions.

Active packet repository. It captures real-time packets exchanged within the ZT-Gym environment. The packets are captured using the "tcpdump" tool and are stored in a MongoDB database. These packets are stored in real time and the database is updated constantly to accommodate any new packets generated or received by the system.

Network Actions Monitoring. Implemented as a Python script with administrative privileges within the ZT-Gym environment, this module monitors network actions specific to the executing

application(s). It focuses on aspects such as port bindings and the transport layer protocols employed for transmission through these ports. Notably, this module replicates the functionality described in the *Host Module* (Section 5). To achieve its monitoring objectives, the script uses the Linux utility `ss -plan` to identify the ports bound by the applications [5]. Subsequently, it transmits a tuple to the packet mapper, encompassing information such as the application name, port numbers in use, destination IP address being communicated with, and the timestamp. Any updates in these features trigger notifications to the Packet Mapper module, facilitating the classification of packets to their respective applications.

Packet Mapper. It plays a crucial role in associating network packets with specific applications. This module resembles part of the functionality of the *Controller-Specific Module* described in Section 6. In this mapping process, five key features are considered: the packet's timestamp, source and destination IP addresses, and source and destination ports. Timestamp is a pivotal parameter for the mapping packets to specific applications running in ZT-Gym. This is because different applications may use the same port(s) at different points in time. To achieve this, the module leverages the timestamp provided by the Network Actions Monitoring module. Packets transmitted from this timestamp onward are attributed to the corresponding application identified in the tuple. In cases in which the module detects another application utilizing the same port at a later time, packets sent or received up until the second application binds the port are categorized under the purview of the first application. The approach ensures accurate packet classification.

Model Training. This module takes the generated datasets, containing benign network data from applications, as input. Its functionality for training the models mirrors that of the *ARL* (Section 7), *RTFSL* (Section 8) and *RGAM* (Section 9) modules, including the model retraining until the user-specified heuristics are met.

Fidelity. Note that every component within the ZT-Gym environment replicates the exact functionality of its counterpart in the ZT-SDN architecture (Figure 1). The key distinction lies in the ZT-Gym environment's controlled setting, where data is generated. In this environment, the ML models are trained offline using the generated datasets. In contrast, ZT-SDN learns directly from data generated and exchanged in the underlying network. Since the applications scrutinized are the same as the ones expected to run in the network, the corresponding packet-level features (e.g., payload) are identical. Some packet-level features, such as TTL, are standardized by the operating systems. However, flow statistics may vary from the actual network due to different conditions, such as lower bandwidth. As demonstrated in Sections 12.3 and 12.4, RTFSL can tolerate reasonable degradations. Administrators can also adjust the bandwidth within the Gym environment [1], allowing for data generation that closely mirrors their networks.

12 Evaluation

We focus on answering the following research questions (RQ):

- **RQ1:** How effective is the ARL module in enforcing network access control?
- **RQ2:** Why is KitNet the preferred learning model for the ARL module, and how do baseline anomaly detection models compare in terms of performance?
- **RQ3:** Once access has been granted, how effective is the RTFSL module in ensuring that the usage of the deployed policies adheres to the learned benign behavior?
- **RQ4:** How do network delays impact the RTFSL model's effectiveness?
- **RQ5:** Is ZT-SDN scalable when applied to an SDN network? Does it degrade the network throughput?

Table 1. Services Used for Evaluation from the MAWI Dataset

Service Port Number	Service	Protocol	Description
80	HTTP	TCP	Hyper Text Transfer Protocol (HTTP): port used for web traffic.
443	HTTPS	TCP	HTTPS / SSL: encrypted web traffic, also used for VPN tunnels over HTTP.
3479	Games	UDP	Microsoft Teams uses UDP ports 3478 through 3481 for media traffic, as well as TCP ports 80 and 443. Apple FaceTime, Apple Game Center use ports 3478-3497 (UDP). Call of Duty World at War. Playstation 4 game ports.
19305	VoIP	UDP	Google Talk, DUO, Hangouts commonly use ports 19302-19308 UDP and 19305-19308 TCP.

12.1 RQ1: ARL Module Effectiveness

The access control aspect concerns the evaluation of our module's ability to correctly allow or deny network accesses during ML module enforcement.

Experiment Setup. We evaluated the effectiveness of the ARL module using two experimental setups: (1) a real-world traffic dataset captured at a major transit link and (2) synthetic traffic generated within Mininet, an SDN environment.

Setup 1 – Real traffic dataset: For the real-world traffic evaluation, we used the MAWI traffic datasets [49], a comprehensive collection of Internet traffic data from the WIDE (Widely Integrated Distributed Environment) project. The dataset, monitored by the MAWI Working Group since 2001, includes packet traces from trans-Pacific links between Japan and the United States. The MAWI dataset is widely recognized as a benchmark in network traffic analysis owing to its longitudinal coverage, diversity of protocols, and high-fidelity capture of real-world traffic and network behavior.

We selected network traffic data from two periods: 2018 (older data) and 2024 (recent data). For each period, we analyzed traffic from prominent protocol stacks, specifically `ETHERNET_IP_TCP` and `ETHERNET_IP_UDP`. Services were chosen based on the following criteria: (1) At least one client user or application connecting to the service. (2) A sufficient volume of packet data for model training and evaluation (minimum of 20K packets). (3) Adequate flow duration to capture transmission behavior (minimum of 10 minutes).

While the MAWI dataset lacks direct information about the applications generating the traffic, we inferred application types by mapping port numbers to services using the SpeedGuide database [44]. SpeedGuide is a comprehensive online resource offering an extensive database of TCP and UDP port numbers. It provides details about well-known, registered, and private port assignments, along with information on associated protocols and services. Table 1 summarizes the selected services, their port numbers, and protocol stacks. We extracted data from four service categories: HTTP, HTTPS, Gaming/Web Calling, and Voice over IP (VoIP). HTTP and HTTPS represent dominant, general-purpose traffic types that account for a significant portion of modern network communication [20]. In contrast, Gaming/Web Calling and VoIP traffic, while less prevalent, exhibit specialized characteristics such as real-time communication and low-latency requirements. We test our approach in both traffic settings.

Setup 2 – Using Mininet: We utilized Mininet to simulate a controlled network environment. A Linux-based server was deployed in one container, whereas a network client was set up in another container, both connected to the same virtual network. The experiments revolved around generating realistic network traffic between these two containers, for which we employed the

`iperf` tool. The client uses two communication channels based on the `ETHERNET_IP_TCP` and another two channels based on the `ETHERNET_IP_UDP` protocol stacks to communicate with the server. Those channels are initiated from a random source port in every session on the client side and directed to ports 5001 (TCP) and 5010 (UDP) on the server side. The application transmits data as fast as possible, leveraging as much available bandwidth as possible.

Model Training. We train edge-specific KitNet models on the constructed CR graph following the process outlined in Section 7. The model parameters were configured as follows: the number of feature mapping samples was set to 200 [33] and the minimum number of training samples was 20K. The maximum reconstruction error, measured as **Root-Mean-Square Error (RMSE)**, was constrained to 0.009 during training and 0.7 during validation.

Model Testing. For model testing, we set the RMSE threshold to 0.7 (the threshold for the validation period). We evaluate the efficacy of the ARL module in three distinct settings:

(1) *Normal Benign Performance:* This setting assesses the module's ability to perform on unseen (future) benign data from the same application running on the same host machine.
(2) *Abnormal Benign Detection:* In this setting, we test whether the module can identify deviations in benign traffic patterns. Specifically, we send benign traffic packets generated by other applications (either of the same or different type) using the same protocol stack to determine whether the model flags them as anomalous. Note that we change the source and destination IP to the ones on which the model was trained. Thus, the module treats the packets as packets sent from the same application on which it has been trained. These deviations are categorized as "abnormal benign" because, while the traffic originates from benign applications, it diverges from the patterns learned by the model.
(3) *Abnormal Malicious Detection:* To evaluate the module's effectiveness in detecting malicious traffic, we focused on two widely recognized attack types: flooding and port scanning. These attacks were selected owing to their prevalence in evaluating anomaly detection systems [18, 37, 51], their significance as common network threats, and their ability to cause notable behavioral deviations in communication flows monitored by ZT-SDN.
These attack types are also highly relevant to access control for several reasons. Port scanning involves probing systems and ports that an entity may not be authorized to access. Similarly, flooding attacks aim to overwhelm resources that the attacker should not have access to. Even if a compromised entity floods a resource to which they are supposed to have access, it abuses the granted network permissions.
The attacks were executed using the Linux tool `hping3`. For port scanning, we used the `--scan` option to target a wide range of ports. For flooding attacks, TCP floods were generated using the `-S --flood` options, whereas UDP floods were generated with the `--flood -2` options.

Packets are identified as normal benign if the RMSE computed by the model is less than the detection threshold (0.7) and considered abnormal if the RMSE threshold is exceeded.

Results. We assess the detection performance of this module for the different attack and benign data, employing the following metrics: **true positives (TPs)**, **true negatives (TNs)**, **false positives (FPs)**, and **false negatives (FNs)**. TP and FN are relevant to the abnormal benign or malicious data, indicating the proportion of examples correctly classified as deviant or mistakenly classified as normal benign. TN and FP pertain to normal benign data, representing the portion of examples correctly classified as normal benign or mistakenly classified as deviant.

Table 2 shows the ARL prediction results for the UDP-based applications/services. The table presents the results in a cross-evaluation manner in which the vertical axis indicates the data on which the model is individually trained. For each set of training data, we report the number of

Table 2. ARL Evaluation Results for UDP-Based Services

Testing (Unseen) → / Training ↓	2018 VoIP 1 (UDP)	2018 VoIP 2 (UDP)	2019 VoIP 3 (UDP)	2024 Game 1 (UDP)	2018 VoIP 4 (UDP)	iperf (udp)	Flooding (UDP)	PortScan (UDP)
2018 VoIP 1 (UDP) **Training stop: 20204** **Max validation score: 0.023**	Total: 203409 TP: 0 TN: 203409 FP: 0 FN: 0 Max Error: 0.023 Min Error: 4.28e-5	Total: 1449 TP: 1449 TN: 0 FP: 0 FN: 0 Max Error: 2.57e18 Min Error: 2.57e18	Total: 1424 TP: 1424 TN: 0 FP: 0 FN: 0 Max Error: 4.9e18 Min Error: 4.9e18	Total: 32872 TP: 32872 TN: 0 FP: 0 FN: 0 Max Error: 8.67e19 Min Error: 8.67e19	Total: 335221 TP: 335221 TN: 0 FP: 0 FN: 0 Max Error: 2.09e19 Min Error: 2.09e19	Total: 35672 TP: 35672 TN: 0 FP: 0 FN: 0 Max Error: 1.22e20 Min Error: 7.21e19	Total: 24753 TP: 24753 TN: 0 FP: 0 FN: 0 Max Error: 2.6e20 Min Error: 7.2e19	Total: 5282 TP: 5282 TN: 0 FP: 0 FN: 0 Max Error: 2.81e20 Min Error: 2.80e20
2024 Game 1 (UDP) **Training stop: 20200** **Max validation score: 0.006**	Total: 170646 TP: 170646 TN: 0 FP: 0 FN: 0 Max Error: 8.677e19 Min Error: 8.67e19	Total: 1449 TP: 1449 TN: 0 FP: 0 FN: 0 Max Error: 8.57e19 Min Error: 8.57e19	Total: 1424 TP: 1424 TN: 0 FP: 0 FN: 0 Max Error: 8.49e19 Min Error: 8.49e19	Total: 39899 TP: 0 TN: 39899 FP: 0 FN: 0 Max Error: .006 Min Error: 4.05e-05	Total: 335221 TP: 335221 TN: 0 FP: 0 FN: 0 Max Error: 9.68e19 Min Error: 9.68e19	Total: 35672 TP: 35672 TN: 0 FP: 0 FN: 0 Max Error: 6.53e19 Min Error: 3.05e19	Total: 24753 TP: 24753 TN: 0 FP: 0 FN: 0 Max Error: 2.16e20 Min Error: 7.73e18	Total: 5282 TP: 5282 TN: 0 FP: 0 FN: 0 Max Error: 2.38e20 Min Error: 2.38e20
2018 VoIP 4 (UDP) **Training stop: 20200** **Max validation score: 0.026**	Total: 170646 TP: 170646 TN: 0 FP: 0 FN: 0 Max Error: 3.37e17 Min Error: 3.37e17	Total: 1449 TP: 1449 TN: 0 FP: 0 FN: 0 Max Error: 2.34e19 Min Error: 2.34e19	Total: 1424 TP: 1424 TN: 0 FP: 0 FN: 0 Max Error: 2.58e19 Min Error: 2.58e19	Total: 32872 TP: 32872 TN: 0 FP: 0 FN: 0 Max Error: 9.68e19 Min Error: 9.68e19	Total: 386575 TP: 0 TN: 386575 FP: 0 FN: 0 Max Error: 0.026 Min Error: 4.28e-5	Total: 35672 TP: 35672 TN: 0 FP: 0 FN: 0 Max Error: 1.4e20 Min Error: 7.28e19	Total: 24753 TP: 24753 TN: 0 FP: 0 FN: 0 Max Error: 7.2e19 Min Error: 2.8e19	Total: 5282 TP: 5282 TN: 0 FP: 0 FN: 0 Max Error: 3.01e20 Min Error: 3e20
iperf (UDP) **Training stop: 20201** **Max validation score: 0.021**	Total: 170646 TP: 170646 TN: 0 FP: 0 FN: 0 Max Error: 7.11e19 Min Error: 7.04e19	Total: 1449 TP: 1449 TN: 0 FP: 0 FN: 0 Max Error: 7.11e19 Min Error: 7.10e19	Total: 1424 TP: 1424 TN: 0 FP: 0 FN: 0 Max Error: 7.11e19 Min Error: 7.1e19	Total: 32872 TP: 32872 TN: 0 FP: 0 FN: 0 Max Error: 1.24e19 Min Error: 7.72e18	Total: 335221 TP: 335221 TN: 0 FP: 0 FN: 0 Max Error: 7.11e19 Min Error: 7.04e19	Total: 304952 TP: 0 TN: 304952 FP: 0 FN: 0 Max Error: 0.07 Min Error: 0.0018	Total: 24753 TP: 24753 TN: 0 FP: 0 FN: 0 Max Error: 1.02e19 Min Error: 1.02e19	Total: 5282 TP: 5282 TN: 0 FP: 0 FN: 0 Max Error: 1.54e20 Min Error: 7.18e18

The "e" notation denotes a power of 10.

training samples supplied to the model as well as the maximum RMSE reported during the validation period. The horizontal axis indicates the data on which the model is tested (unseen traffic data). The orange cells denote the test to evaluate the effectiveness of the model in recognizing normal benign data, whereas the green cells evaluate the effectiveness of the model in recognizing abnormal benign data. Finally, the red cells indicate the effectiveness of the model in recognizing abnormal malicious data. In each cell, we report the total number of packets on which the model was tested as well as the TP, TN, FP, and FN numbers. We also report the minimum and maximum RMSE errors reported by the model while testing in those packets. The column/row naming follows the following convention: "[Dataset Year] [Application Type] [Client ID] ([Protocol])." Attack traffic data and `iperf` do not have a year as they were generated using tools. Note that the "e" notation denotes a power of 10.

Our findings demonstrate that the module achieves a 100% accuracy in identifying normal benign traffic, effectively learning the distribution for each of the CR graph edges with just 20,000 training samples. Furthermore, the model exhibits a 100% accuracy in detecting anomalous benign packets, showcasing its ability to discern differences in protocol header values even when client applications connect to the same service (e.g., VoIP-to-VoIP communication), therefore utilizing the same destination ports. Last, the model maintains 100% accuracy in detecting anomalous malicious traffic. In these scenarios, the port number is a key feature, as malicious activities such as flooding and port scanning rely on a range of source or destination ports not utilized by the benign applications on which the model is trained.

Tables 3 and 4 show the ARL performance results for TCP-based applications/services. As the tables show, the ARL training requires more packet samples from each application as the TCP protocol stack contains more features compared with the UDP stack. Thus, more training data is required for our unsupervised model to understand the relation between header–value pairs. The results are consistent with the ones presented in Table 2. The ARL module can distinguish between normal benign and abnormal malicious or abnormal benign accesses.

Table 3. ARL Evaluation Results for TCP-based Services

Testing (Unseen) → / Training ↓	2018 HTTP 1 (TCP)	2018 HTTP 3 (TCP)	2018 HTTPS 1 (TCP)	2018 HTTPS 2 (TCP)	2024 HTTP 1 (TCP)	2024 HTTP 2 (TCP)
2018 HTTP 1 (TCP) **Training stop: 30200** **Max validation score: 0.03**	Total: 200000 TP: 0 TN: 200000 FP: 0 FN: 0 Max Error: 0.39 Min Error: 0.60	Total: 200000 TP: 200000 TN: FP: 0 FN: 0 Max Error: 1.04e16 Min Error: 8.4e15	Total: 80796 TP: 80796 TN: FP: 0 FN: 0 Max Error: 1.56e18 Min Error: 1.56e18	Total: 26107 TP: 26107 TN: 0 FP: 0 FN: 0 Max Error: 1.56e18 Min Error: 1.56e18	Total: 134195 TP: 134195 TN: FP: 0 FN: 0 Max Error: 2.20e16 Min Error: 2.12e16	Total: 22984 TP: 22984 TN: 0 FP: 0 FN: 0 Max Error: 2.59e17 Min Error: 2.59e17
2018 HTTPS 1 (TCP) **Training stop: 30200** **Max validation score: 0.13**	Total: 199999 TP: 199999 TN: 0 FP: 0 FN: 0 Max Error: 7.32e19 Min Error: 5.51e19	Total: 485192 TP: 485192 TN: 0 FP: 0 FN: 0 Max Error: 2.41e20 Min Error: 2.55e18	Total: 88528 TP: 0 TN: 88528 FP: 0 FN: 0 Max Error: 0.13 Min Error: 0.004	Total: 26107 TP: 26107 TN: 0 FP: 0 FN: 0 Max Error: 3.66e19 Min Error: 3.65e19	Total: 134195 TP: 134195 TN: 0 FP: 0 FN: 0 Max Error: 8.21e19 Min Error: 8.78e18	Total: 22984 TP: 22984 TN: 0 FP: 0 FN: 0 Max Error: 3.18e19 Min Error: 3.06e19
2024 HTTP 1 (TCP) **Training stop: 30200** **Max validation score: 0.15**	Total: 199999 TP: 199999 TN: 0 FP: 0 FN: 0 Max Error: 2.46e16 Min Error: 2.46e16	Total: 485192 TP: 485192 TN: 0 FP: 0 FN: 0 Max Error: 2.65e16 Min Error: 2.65e16	Total: 80796 TP: 80796 TN: 0 FP: 0 FN: 0 Max Error: 1.81e18 Min Error: 1.81e18	Total: 26107 TP: 26107 TN: 0 FP: 0 FN: 0 Max Error: 1.81e18 Min Error: 1.81e18	Total: 1170 TP: 0 TN: 1170 FP: 0 FN: 0 Max Error: 0.59 Min Error: 0.21	Total: 22984 TP: 22984 TN: 0 FP: 0 FN: 0 Max Error: 3.25e17 Min Error: 3.25e17
2024 HTTPS 2 (TCP) **Training stop: 100200** **Max validation score: 0.011**	Total: 199999 TP: 199999 TN: 0 FP: 0 FN: 0 Max Error: 1.59e18 Min Error: 1.59e18	Total: 485192 TP: 485192 TN: 0 FP: 0 FN: 0 Max Error: 1.59e18 Min Error: 1.59e18	Total: 80796 TP: 80796 TN: 0 FP: 0 FN: 0 Max Error: 8.8e15 Min Error: 8.8e15	Total: 26107 TP: 26107 TN: 0 FP: 0 FN: 0 Max Error: 4.3e15 Min Error: 2428.37	Total: 134195 TP: 134195 TN: 0 FP: 0 FN: 0 Max Error: 1.59e18 Min Error: 1.59e18	Total: 22984 TP: 22984 TN: 0 FP: 0 FN: 0 Max Error: 1.59e18 Min Error: 1.59e18
iperf (TCP) **Training stop: 30200** **Max validation score: 0.014**	Total: 199999 TP: 199999 TN: 0 FP: 0 FN: 0 Max Error: 2.28e19 Min Error: 2.29e19	Total: 485192 TP: 485192 TN: 0 FP: 0 FN: 0 Max Error: 4.18e16 Min Error: 4.18e16	Total: 80796 TP: 80796 TN: 0 FP: 0 FN: 0 Max Error: 2.11e19 Min Error: 2.11e19	Total: 26107 TP: 26107 TN: 0 FP: 0 FN: 0 Max Error: 2.12e19 Min Error: 2.11e19	Total: 134195 TP: 134195 TN: 0 FP: 0 FN: 0 Max Error: 2.29e19 Min Error: 2.28e19	Total: 22984 TP: 22984 TN: 0 FP: 0 FN: 0 Max Error: 2.29e19 Min Error: 2.28e19

Model Interpretability. The results demonstrate that a KitNet model trained on CR graph edges effectively learns allowed protocol headers, particularly static fields such as protocol type, service port numbers, and protocol flags. The model is highly sensitive to deviations: for example, if an endpoint in the CR graph communicates with unauthorized ports or systems, it generates an RMSE reconstruction error exceeding the anomaly threshold. Another example is the case of TCP communications, which rarely set the RST flag unless an issue arises; deviations from this normal behavior are detected if RST is absent from the baseline profile.

Notably, the model can detect abnormalities even in benign scenarios within the same application domain (e.g., identical port sets) provided at least one header value, such as TTL, varies across end systems. However, if a test client exhibits packet headers identical to the training data, the model may not detect the difference. This, however, is not a problem as it aligns with ARL's objective: learning the allowed protocol stack. Future work will explore the importance of specific protocol headers in model interpretation and their impact on reconstruction error through an ablation study.

12.2 RQ2: Comparing KitNet Performance With Other Anomaly Detection Models

In this section, we present an evaluation that supports the selection of KitNet as the model of choice for the implementation of the ARL module.

We trained several baseline anomaly detection models to evaluate their performance on the individual edges of our CR graph model, including Isolation Forest, One-Class SVM, and the Gaussian

Table 4. ARL Evaluation Results for TCP-Based Services

Testing (Unseen) → / Training ↓	2024 HTTPS 1 (TCP)	2024 HTTPS 2 (TCP)	iperf (TCP)	SYN Flooding (TCP)	PortScan (TCP)
2018 HTTP 1 (TCP) **Training stop: 30200** **Max validation score: 0.03**	Total: 866065 TP: 866065 TN: 0 FP: 0 FN: 0 Max Error: 1.54e18 Min Error: 1.54e18	Total: 256112 TP: 256112 TN: 0 FP: 0 FN: 0 Max Error: 1.54e18 Min Error: 1.54e18	Total: 163917 TP: 163917 TN: 0 FP: 0 FN: 0 Max Error: 2.09e19 Min Error: 2.09e19	Total: 375933 TP: 375933 TN: 0 FP: 0 FN: 0 Max Error: 2.09e19 Min Error: 2.09e19	Total: 1993 TP: 1993 TN: 0 FP: 0 FN: 0 Max Error: 2.51e19 Min Error: 2.09e19
2018 HTTPS 1 (TCP) **Training stop: 30200** **Max validation score: 0.13**	Total: 866065 TP: 866065 TN: 0 FP: 0 FN: 0 Max Error: 1.44e20 Min Error: 1.44e20	Total: 256112 TP: 256112 TN: 0 FP: 0 FN: 0 Max Error: 2.21e20 Min Error: 2.21e20	Total: 163917 TP: 163917 TN: 0 FP: 0 FN: 0 Max Error: 2.82e19 Min Error: 2.81e19	Total: 375933 TP: 375933 TN: 0 FP: 0 FN: 0 Max Error: 1.93e19 Min Error: 2.73e20	Total: 1993 TP: 1993 TN: 0 FP: 0 FN: 0 Max Error: 2.62e20 Min Error: 2.62e20
2024 HTTP 1 (TCP) **Training stop: 30200** **Max validation score: 0.15**	Total: 866065 TP: 866065 TN: 0 FP: 0 FN: 0 Max Error: 1.79e18 Min Error: 1.79e18	Total: 256112 TP: 256112 TN: 0 FP: 0 FN: 0 Max Error: 1.79e18 Min Error: 1.79e18	Total: 163917 TP: 163917 TN: 0 FP: 0 FN: 0 Max Error: 2.42e19 Min Error: 2.42e19	Total: 375933 TP: 375933 TN: 0 FP: 0 FN: 0 Max Error: 2.42e19 Min Error: 2.42e19	Total: 1993 TP: 1993 TN: 0 FP: 0 FN: 0 Max Error: 2.92e19 Min Error: 2.42e19
2024 HTTPS 2 (TCP) **Training stop: 100200** **Max validation score: 0.011**	Total: 866065 TP: 866065 TN: 0 FP: 0 FN: 0 Max Error: 4.3e15 Min Error: 1015	Total: 70000 TP: 0 TN: 70000 FP: 0 FN: 0 Max Error: 0.22 Min Error: 0.003	Total: 163917 TP: 163917 TN: 0 FP: 0 FN: 0 Max Error: 2e19 Min Error: 2e19	Total: 375933 TP: 375933 TN: 0 FP: 0 FN: 0 Max Error: 2e19 Min Error: 2e19	Total: 1993 TP: 1993 TN: 0 FP: 0 FN: 0 Max Error: 2.44e19 Min Error: 2e19
iperf (TCP) **Training stop: 30200** **Max validation score: 0.014**	Total: 866065 TP: 866065 TN: 0 FP: 0 FN: 0 Max Error: 2.12e19 Min Error: 2.11e19	Total: 256112 TP: 256112 TN: 0 FP: 0 FN: 0 Max Error: 2.11e19 Min Error: 2.11e19	Total: 133717 TP: 0 TN: 133717 FP: 0 FN: 0 Max Error: 0.07 Min Error: 0.0002	Total: 375933 TP: 375933 TN: 0 FP: 0 FN: 0 Max Error: 5.17e17 Min Error: 3.8e15	Total: 1993 TP: 1993 TN: 0 FP: 0 FN: 0 Max Error: 4.63e18 Min Error: 3.9e15

Mixture Model. All models, including KitNet, were trained on the same dataset and evaluated on the same test (unseen) data.

Table 5 summarizes the performance of these models, including KitNet, across four real traffic datasets derived from the MAWI traffic trace. The metric TN represents the number of instances correctly classified as normal benign traffic, whereas FP denotes the number of legitimate instances incorrectly flagged as anomalous. The results indicate that all baseline models exhibit suboptimal performance compared with KitNet. Notably, KitNet consistently achieved zero false positives, ensuring that no legitimate access requests are denied by the ARL module.

12.3 RQ3: RTFSL Module Effectiveness

Experiment Setup. We used the same experimental setup as in Section 12.1.

Model Training. We construct edge-specific $\{\Delta x_t\}_{t=2}^{n}$ time series models on the computed CR graph following the process outlined in Section 8. The model parameters are configured as follows: the minimum number of training flow statistics samples is set to 150, with a sampling rate of 5 seconds. For anomaly detection, the maximum Euclidean distance threshold is set to 0.8. Finally,

Table 5. Comparison of Our Model of Choice for the ARL Module Against Other Anomaly Detection Candidate Models

Model	2018 VoIP 1 (UDP) Unseen Data Size: 203409	2024 Game 1 (UDP) Unseen Data Size: 39899	2018 VoIP 4 (UDP) Unseen Data Size: 386575	2024 HTTP 1 (TCP) Unseen Data Size: 1170
CR + Isolation Forest	TN: 187694 FP: 15715	TN: 39899 FP: 0	TN: 383428 FP: 3147	TN: 1093 FP: 77
CR + One-Class SVM	TN: 196637 FP: 6775	TN: 39619 FP: 280	TN: 384843 FP: 1732	TN: 0 FP: 1170
CR + Gaussian Mixture	TN: 193260 FP: 10149	TN: 37947 FP: 1952	TN: 367574 FP: 19001	TN: 1111 FP: 59
CR + KitNet (Ours)	**TN: 203409** **FP: 0**	**TN: 39899** **FP: 0**	**TN: 386575** **FP: 0**	**TN: 1170** **FP: 0**

the window size is configured to 70 samples for UDP-based communications and 90 samples for TCP-based communications.

Model Testing. We evaluate the efficacy of the RTFSL module in ensuring that access granted to endpoints is used in accordance with the learned transmission behavior from the training process. In this experimental setup, the ARL module initially grants communication access to endpoints. However, during the enforcement phase, the endpoints deviate from their normal behavior, either adopting the transmission patterns of another benign application or exhibiting malicious behavior.

It is important to note that the model training and enforcement phases were conducted using different traffic trace datasets captured during different periods in the MAWI dataset. These capture periods experienced natural throughput fluctuations due to the starting and stopping of flows. As a result, during testing, the model attempts to match transmission patterns that may have varied due to these throughput fluctuations. Therefore, we evaluate the performance of the RTFSL module in recognizing normality or abnormality in traffic patterns affected by the dynamic fluctuations of network conditions.

Consistent with the evaluation objectives for the ARL module, we assess the RTFSL module's ability to (1) recognize normal benign transmission patterns and (2) detect deviations classified as abnormal benign or malicious behaviors.

Results. Figures 9 and 10 show the evaluation results for the UDP-based flows; Figures 11 and 12 show the evaluation results for the TCP-based flows.

Similar to the previous section, we present the results in a cross-evaluation manner in which the vertical axis indicates the data on which the model is individually trained. For each set of training data, we report the number of training samples supplied to the model. The horizontal axis indicates the data on which the model is tested.

Each cell in the evaluation results shows a plot of how the anomaly error (i.e., DTW Euclidean distance) progresses with the arrival of flow statistics samples for the packets (green curve) and bytes (orange curve). The dotted blue line denotes the moment when the sliding window starts (i.e., the number of collected samples equals the window length). The horizontal dotted red line denotes the anomaly detection threshold (i.e., 0.8). The anomaly detection occurs when a curve meets the anomaly detection boundary. Note that if the green and orange curves are tangent to one another, only the orange curve will be visible in the plot, as it overlays the green curve. Plots in orange boxes pertain to normal benign tests, green to abnormal benign, and red to abnormal malicious.

In both types of flows, we make the following observations: (1) The DTW Euclidean distance for normal benign patterns typically remains far below the anomaly threshold. (2) The module detects both abnormal benign and malicious traffic patterns before the sliding window begins. In

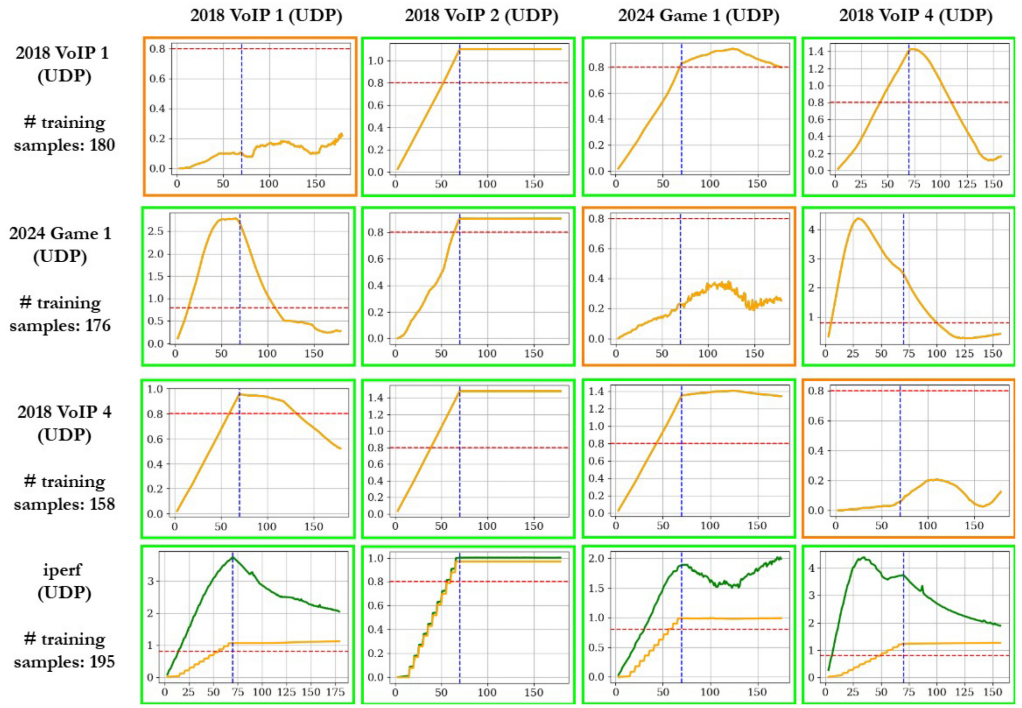


Fig. 9. RTFSL module evaluation results for the UDP-based flows.

other words, in all cases, the module detected the abnormality by collecting statistics samples less than the window length (i.e., in the first couple of minutes of data transmission). For example, the model trained on the "2024 Game 1 (UDP)" application data and tested on "2018 VoIP 1 (UDP)" detected the anomalous flow within 10 samples (less than 1 minute of data transmission) from the flow start (see Figure 9).

12.4 RQ4: RTFSL Effectiveness in Changing Network Conditions

We now explore a distinct scenario in which the training data is generated under network conditions that differ from those present during model testing. This analysis aims to examine the module's robustness to variations in network conditions, specifically bandwidth, link delays, and jitter.

It is worth noting that the experiments conducted with the MAWI datasets (Section 12.3) inherently include fluctuations in these network condition attributes over time due to the thousands of flows traversing the transit link from which the data was captured. In this scenario, we intentionally introduce significant changes to these conditions to further stress-test the module's adaptability.

Experiment Setup. We use the experiment setup 2 discussed in Section 12.1.

Model Training. We train the module with data generated in a network with a bandwidth of 55 Mbps between the communicating entities and no observable link delays or jitter. In contrast, during model enforcement, we enforce the model in a network with significantly lower bandwidth (i.e., 20 Mbps) and progressively introduce link delays and jitter. Our investigation focuses on two key aspects: (1) assessing whether normal benign transmissions are still accurately identified as

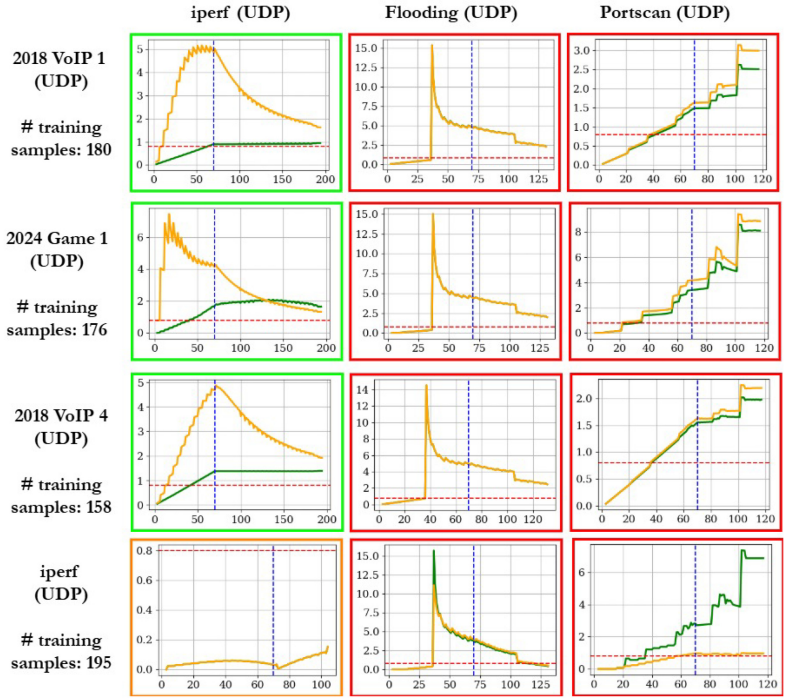


Fig. 10. RTFSL module evaluation results for the UDP-based flows (continued).

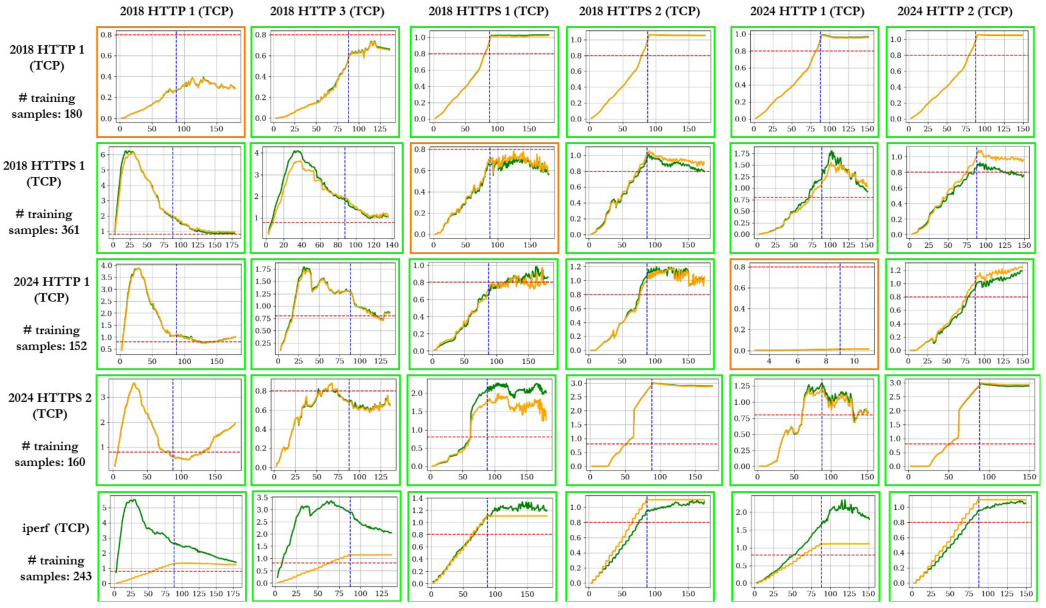


Fig. 11. RTFSL module evaluation results for the TCP-based flows.

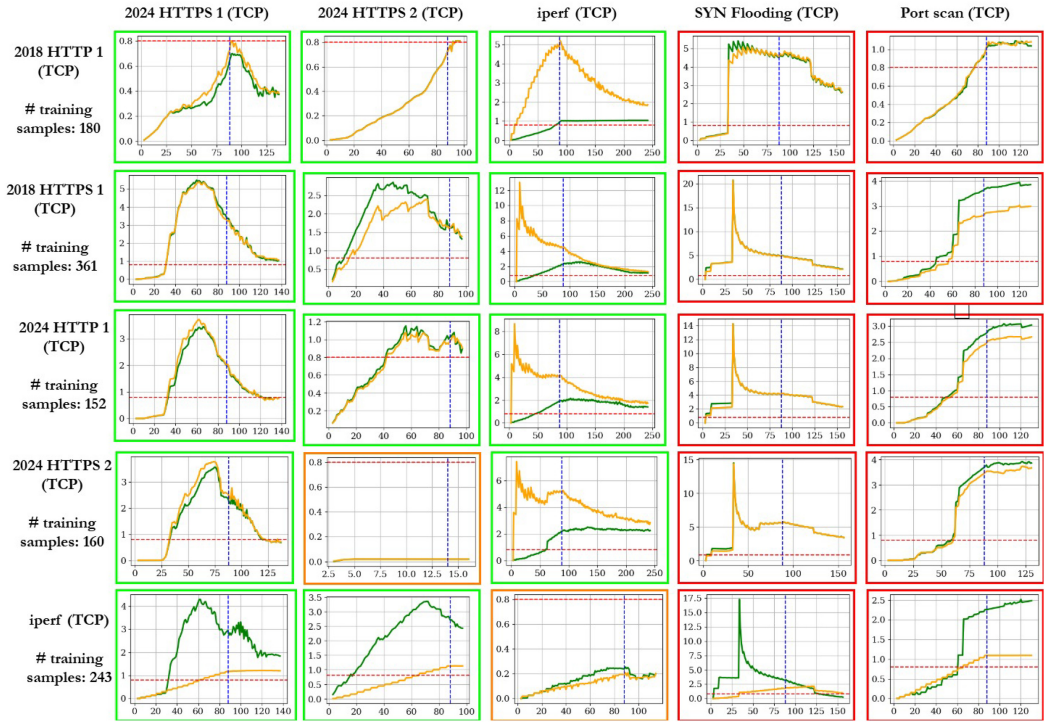


Fig. 12. RTFSL module evaluation results for the TCP-based flows (continued).

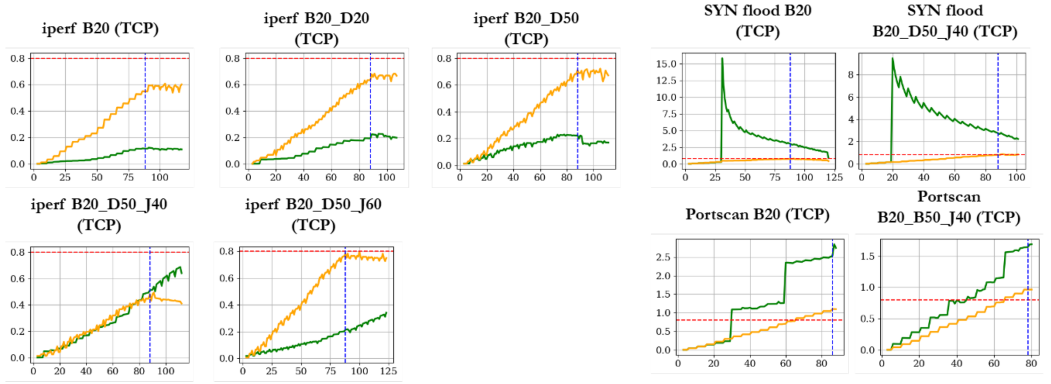


Fig. 13. RTFSL module evaluation results on different training and enforcement network conditions for the TCP-based flows.

benign, thereby preventing degradation of **quality of service (QoS)** by suspending benign flows; and (2) evaluating the model's ability to detect abnormal malicious flows under these changing conditions.

Results. The results of the experiments are presented in the Figures 13 and 14. The name of each plot denotes the traffic patterns on which the module is tested as well as the network conditions. For instance, plot "iperf B20_D50_J40," pertains to the exhibited transmission patterns of iperf when the bandwidth is 20 Mbps with link delays of 50 ms and jitter of 40 ms.

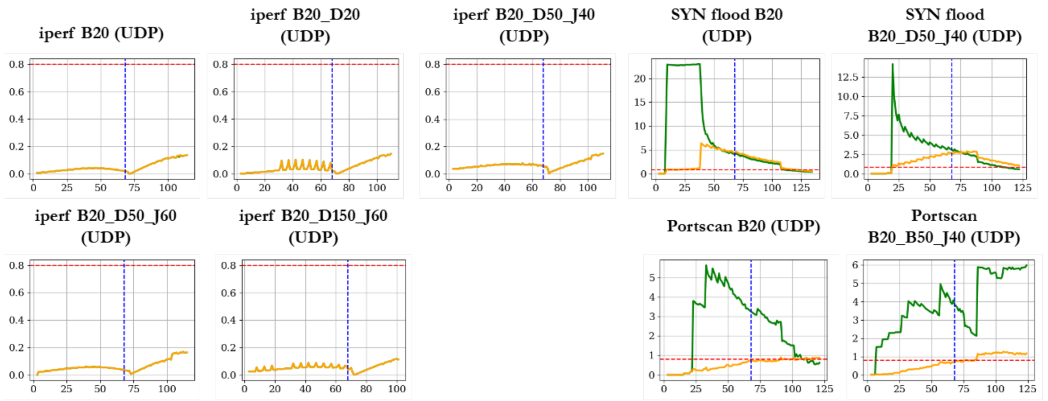


Fig. 14. RTFSL module evaluation results on different training and testing network conditions for the UDP-based flows.

For both TCP- and UDP-based communication, the model demonstrates resilience against a reduction in bandwidth of at least 64%. Specifically, the model exhibits no instances of FP or FN for either benign or anomalous malicious traffic under these conditions.

Subsequently, we introduce a link delay of 50 ms along with a jitter of up to 60 ms. In the case of TCP-based communication, the model identifies anomalies in the byte time series (orange curve), having an anomaly error of 0.8. Consequently, ZT-SDN terminates the connection. It is noteworthy that network delay and jitter of such magnitude significantly diminish the QoS, even without ZT-SDN intervention. Typically, applications with high bandwidth requirements or real-time communications, such as VoIP, are unable to tolerate jitter beyond 50 ms [12, 32]. Thus, the communication, even without ZT-SDN, would be impractical in such scenarios.

Conversely, in the case of the UDP, the model remains robust even in the presence of extremely high delay and jitter. This resilience can be attributed to the simplicity of the UDP, which lacks congestion control and retransmission mechanisms found in the TCP. Consequently, the shape of the time series in these situations does not undergo notable changes that would trigger anomalies in ZT-SDN. Last, the abnormal malicious patterns exhibited by the flooding and port scanning are detectable in the reduced bandwidth setting. The model continues to exhibit robustness in detecting these attacks beyond 50 ms in both delay and jitter.

12.5 RQ5: Scalability of ZT-SDN

We evaluate the performance of ZT-SDN by comparing it to the baseline provided by the ONOS's reactive forwarding application, Fwd. Our evaluation focuses on three key metrics: (1) the number of control plane invocations (i.e., `PACKET_IN` messages) from the data plane, (2) the achievable bandwidth in the data plane as a result of control plane processing, and (3) the ZT-SDN's control plane processing delay.

Experiment Setup. We set up a network of varying sizes in terms of hosts, switches, and the number of switches between communicating entities. Both the endpoints and switches run individually in Linux containers within the Mininet environment. We utilize `Open vSwitches` [40] for the switches. To generate traffic and measure the maximum achievable bandwidth between each client and server machine, we use the `iperf` tool in bandwidth testing mode. In this mode, `iperf` uses two bidirectional channels and sends packets rapidly to stress test the connection and measure the bandwidth. In these experiments, we test the scalability of our approach under the most stressful network conditions: (1) all client machines start their applications approximately

Table 6. Number of Packet-In, Rules Per Switch, and Total Processing Times

Topology (Hosts x Switches)	Total Packet-In		Number of Rules Per Switch		Total Processing Time (sec)	
	ZT-SDN	Fwd	ZT-SDN	Fwd	ZT-SDN	Fwd
2H x 4 SW	15	76	2	4	0.15	0.07
4H x 4 SW	73	138	6	12	0.85	0.12
10 H x 5 SW	152	449	18	36	1.07	0.47
20 H x 10 SW	596	1745	38	76	2.77	0.82
30 H x 10 SW	758	2198	58	116	4.37	0.83
40 H x 15 SW	1900	3624	78	153	6.27	0.89
100 H x 15 SW	7378	19588	198	396	20.98	2.68
150 H x 15 SW	16441	36836	298	596	30.28	3.06

at the same time and (2) the communication path between entities is always the longest possible, traversing all network switches.

Model Training. We train ARL and RTFSL similarly to the methods described in Sections 12.1 and 12.3, respectively. Using our RGAM approach, we mine rules from CR graph edge-specific datasets generated by the CSM. The datasets comprise approximately a total of 35,000 traffic samples from application execution. RGAM generated two flow rules for each communication endpoint: r1 = [dstAddr = *SERVER_ADDR*, dstPort = 5001, dscp = 0, ecn = 0, etherType = 2048, proto = TCP, srcAddr = *CLIENT_ADDR*, vlanID= -1], and r2 = [destAddr = *CLIENT_ADDR*, dscp = 0, ecn = 0, etherType = 2048, proto = TCP, srcAddr = *SERVER_ADDR*, srcPort = 5001, vlanID = -1]. Here, *SERVER_ADDR* and *CLIENT_ADDR* represent the IPv4 addresses of the server and each client, respectively. According to our model analysis, the source port on any client machine cannot be predicted; thus, it is excluded from the generated rules. Additionally, the analysis indicates that these two rules have a 100% association score.

Results. Table 6 shows the scalability performance results comparing the Fwd with ZT-SDN. The topology column reports the number of host machines (clients and server) and the number of switches in the path between each client and the server. Packet-In indicates the number of times the controller was invoked due to the absence of rules in the data plane. Number of rules expresses the number of rules installed at the switches by each approach. Note that Fwd generates rules dynamically at run time based on `PACKET_IN` message headers, which include source and destination addresses, ports, and transport layer protocol. Finally, the total processing time represents the time required for ZT-SDN and Fwd to process all the access requests (i.e., `PACKET_IN`). For ZT-SDN, this includes the processing time of both the CSM and ML modules (encompassing packet header processing and model inferences). It is important to note that Fwd does not enforce access control and allows all accesses.

In all cases, ZT-SDN demonstrates a reduction in the number of `PACKET_IN` messages compared with Fwd owing to ZT-SDN's proactive rule deployment approach. Finally, the number of rules installed in the network is reduced by 50% in ZT-SDN compared with rules installed by Fwd. This is because Fwd enforces rules with both the source and destination ports. Since, in this scenario, there are two bidirectional channels, the number of rules enforced by Fwd at each switch would be double the number of rules enforced by ZT-SDN.

Table 7 shows the `iperf` reported average throughput and the amount of data transmitted per host across the same network topologies reported in Table 6. The results suggest that ZT-SDN does not cause network degradation despite the additional access request processing overhead at the

Table 7. Average Maximum Achievable Bandwidth and Bytes Transmitted Between the Communicating Entities

Topology (Hosts x Switches)	Average Throughput Per Client (Gbps)		Average Bytes Transmitted Per Client (GBytes)	
	ZT-SDN	Fwd	ZT-SDN	Fwd
2H x 4 SW	49.5	52.9	173	185
4H x 4 SW	44.16	43.43	154.3	151.66
10 H x 5 SW	31.29	31.07	109.22	108.44
20 H x 10 SW	16.27	16.12	56.85	56.29
30 H x 10 SW	11.86	12.15	41.44	42.47
40 H x 15 SW	9.20	9.16	32.15	32.02
100 H x 15 SW	3.63	3.15	13.66	12.2
150 H x 15 SW	2.59	2.55	9.91	9.77

control plane. In fact, as the network topology scales, ZT-SDN exhibits performance equivalent to the baseline.

13 Conclusion and Future Work

In this article, we have introduced ZT-SDN, a novel end-to-end ZT architecture for SDNs. ZT-SDN implements automated learning processes from the underlying network that (1) capture the communication requirements of entities in the network, (2) derive the access control rules allowing entities to execute their missions successfully in the least privilege, and (3) learn the data transmission behavior of the entities using those granted network permissions by extracting their communication patterns. Additionally, ZT-Gym is introduced to facilitate the offline generation of application datasets and training of ML modules, ensuring benign training datasets in scenarios in which network data transmissions are not guaranteed to be benign. Our experimental results demonstrate that ZT-SDN effectively prevents unauthorized access to network resources and detects deviant behaviors in entities using permitted flows. Furthermore, we show that ZT-SDN exhibits reasonable tolerance to changing network conditions. We also present the scalability performance of our framework across various topology sizes, comparing it against the ONOS baseline reactive forwarding.

Future Work. Future research directions for ZT-SDN focus on addressing key challenges and advancing the capabilities of the framework.

First, the current implementation trains models on the individual edges of the CR graph in both the ARL and RTFSL modules. However, as the CR graph grows in complexity with hundreds of edges, training time and model management become increasingly resource intensive. To address this, our aim is to develop a graph-pruning strategy that simplifies the CR graph. One potential approach is to identify and unify nodes that exhibit similar behaviors, such as users of the same application instance with consistent usage patterns. This unification would reduce redundant edges and training overhead.

Second, we plan to explore methods to relax the assumption of training data that are benign only. This involves designing an automated data cleaning pipeline that identifies and excludes anomalous examples from the training dataset, ensuring robust model performance even in scenarios with abnormal data. This approach could also reduce the overhead associated with running applications in the ZT-Gym for offline dataset generation.

In addition, our goal is to implement an automated model relearning mechanism to adapt to evolving benign network behaviors. For example, when an application is updated to introduce

new capabilities—potentially generating new traffic flows—ZT-SDN would automatically update its ML models. This adaptation will involve granting new access permissions, recognizing updated flow behaviors, and generating the corresponding firewall rules to seamlessly accommodate the new requirements.

Finally, we aim to scale the ZT-SDN architecture for much larger and more complex networks. In networks with hundreds or thousands of nodes (e.g., user hosts, network switches, servers), a distributed control plane is typically adopted, in which multiple controllers manage different segments of the data plane. This setup reduces the workload on individual controllers and improves overall scalability. We plan to explore how ZT-SDN can be extended to effectively support distributed control plane architectures, enabling its deployment in large-scale network environments.

Acknowledgments

The authors would like to thank Sumedh Nitin Joshi for his assistance with the implementation of the ZT-Gym. They also thank the anonymous reviewers, as well as Adrian Shuai Li (Purdue University) and Kevin Chuanpu Fu (Tsinghua University), for their insightful feedback and constructive comments on the manuscript.

References

[1] [n. d.]. *tc(8) - Linux Manual Page*. Retrieved from https://man7.org/linux/man-pages/man8/tc.8.html

[2] 2015. *OpenFlow Switch Specification (version 1.5.1)*. Technical Report. Open Networking Foundation.

[3] Zakaria Abou El Houda, Abdelhakim Senhaji Hafid, and Lyes Khoukhi. 2021. A novel machine learning framework for advanced attack detection using SDN. In *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 1–6.

[4] Rakesh Agarwal, Ramakrishnan Srikant, et al. 1994. Fast algorithms for mining association rules. In *Proc. of the 20th VLDB Conference*, Vol. 487. 499.

[5] Alexey Kuznetsov and Michael Prokop. [n. d.]. ss(8) - linux manual page. *Linux Documentation* ([n. d.]). https://linux.die.net/man/8/ss

[6] Iffat Anjum, Daniel Kostecki, Ethan Leba, Jessica Sokal, Rajit Bharambe, William Enck, Cristina Nita-Rotaru, and Bradley Reaves. 2022. Removing the reliance on perimeters for security using network views. In *Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies*. 151–162.

[7] Iffat Anjum, Jessica Sokal, Hafiza Ramzah Rehman, Ben Weintraub, Ethan Leba, William Enck, Cristina Nita-Rotaru, and Bradley Reaves. 2023. MSNetViews: Geographically distributed management of enterprise network security policy. In *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies*. 121–132.

[8] Daniele Apiletti, Elena Baralis, Tania Cerquitelli, and Vincenzo D'Elia. 2009. Characterizing network traffic by means of the NetMine framework. *Computer Networks* 53, 6 (2009), 774–789.

[9] Luiz Fernando Carvalho, Taufik Abrão, Leonardo de Souza Mendes, and Mario Lemes Proença Jr. 2018. An ecosystem for anomaly detection and mitigation in software-defined networking. *Expert Systems with Applications* 104 (2018), 121–133.

[10] Martin Casado, Michael J. Freedman, Justin Pettit, Jianying Luo, Nick McKeown, and Scott Shenker. 2007. Ethane: Taking control of the enterprise. *ACM SIGCOMM Computer Communication Review* 37, 4 (2007), 1–12.

[11] Cisco. [n. d.]. *Configuring Traffic Mirroring*. Retrieved from https://www.cisco.com/c/en/us/td/docs/iosxr/ncs5000/interfaces/711x/configuration/guide/b-interfaces-hardware-component-cg-ncs5000-711x/configuring-traffic-mirroring.pdf

[12] Cisco. 2020. *What is Jitter?* https://documentation.meraki.com/MR/Wi-Fi_Basics_and_Best_Practices/What_is_Jitter%3F

[13] Levente Csikor, Sriram Ramachandran, and Anantharaman Lakshminarayanan. 2022. ZeroDNS: Towards better zero trust security using DNS. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 699–713.

[14] Anderson Santos da Silva, Juliano Araujo Wickboldt, Lisandro Zambenedetti Granville, and Alberto Schaeffer-Filho. 2016. ATLANTIC: A framework for anomaly traffic detection, classification, and mitigation in SDN. In *NOMS 2016-2016 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 27–35.

[15] Mahmoud Said El Sayed, Nhien-An Le-Khac, Marianne A. Azer, and Anca D. Jurcut. 2022. A flow-based anomaly detection approach with feature selection method against DDoS attacks in SDNs. *IEEE Transactions on Cognitive Communications and Networking* 8, 4 (2022), 1862–1880.

[16] Paul Emmerich, Maximilian Pudelko, Sebastian Gallenmüller, and Georg Carle. 2017. FlowScope: Efficient packet capture and storage in 100 Gbit/s networks. In *2017 IFIP Networking Conference (IFIP Networking) and Workshops*. IEEE, 1–9.

[17] Sahil Garg, Kuljeet Kaur, Neeraj Kumar, and Joel J. P. C. Rodrigues. 2019. Hybrid deep-learning-based anomaly detection scheme for suspicious flow detection in SDN: A social multimedia perspective. *IEEE Transactions on Multimedia* 21, 3 (2019), 566–578.

[18] Kostas Giotis, Christos Argyropoulos, Georgios Androulidakis, Dimitrios Kalogeras, and Vasilis Maglaris. 2014. Combining OpenFlow and sFlow for an effective and scalable anomaly detection and mitigation mechanism on SDN environments. *Computer Networks* 62 (2014), 122–136.

[19] Korosh Golnabi, Richard K, Min, Latifur Khan, and Ehab Al-Shaer. 2006. Analysis of firewall policy rules using data mining techniques. In *2006 IEEE/IFIP Network Operations and Management Symposium (NOMS 2006)*. IEEE, 305–315.

[20] MAWI Working Group. 2024. MAWI Traffic Trace Dataset of June 19th, 8.45 am 2024. Retrieved January 4, 2025 from https://mawi.wide.ad.jp/mawi/ditl/ditl2024/202406190845.html

[21] Jordan Holland, Paul Schmitt, Nick Feamster, and Prateek Mittal. 2021. New directions in automated traffic analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3366–3383.

[22] Babangida Isyaku, Mohd Soperi Mohd Zahid, Maznah Bte Kamat, Kamalrulnizam Abu Bakar, and Fuad A. Ghaleb. 2020. Software defined networking flow table management of openflow switches performance and security challenges: A survey. *Future Internet* 12, 9 (2020), 147.

[23] Liu Jian-Ping, Liu Juan-Juan, and Wang Dong-Long. 2012. Application analysis of automated testing framework based on robot. (2012), 194–197. https://doi.org/10.1109/ICNDC.2012.53

[24] Charalampos Katsis, Fabrizio Cicala, Dan Thomsen, Nathan Ringo, and Elisa Bertino. 2021. Can I reach you? Do I need to? New semantics in security policy specification and testing. In *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies*. 165–174.

[25] Charalampos Katsis, Fabrizio Cicala, Dan Thomsen, Nathan Ringo, and Elisa Bertino. 2022. Neutron: A graph-based pipeline for zero-trust network architectures. In *Proceedings of the 12th ACM Conference on Data and Application Security and Privacy*. 167–178.

[26] Diego Kreutz, Fernando M. V. Ramos, Paulo Esteves Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. 2014. Software-defined networking: A comprehensive survey. *Proc. IEEE* 103, 1 (2014), 14–76.

[27] Trupti A. Kumbhare and Santosh V. Chobe. 2014. An overview of association rule mining algorithms. *International Journal of Computer Science and Information Technologies* 5, 1 (2014), 927–930.

[28] Willian Tessaro Lunardi, Martin Andreoni Lopez, and Jean-Pierre Giacalone. 2022. Arcade: Adversarially regularized convolutional autoencoder for network anomaly detection. *IEEE Transactions on Network and Service Management* 20, 2 (2022), 1305–1318.

[29] Roberto Maestre. 2025. FastDTW. Retrieved January 9, 2025 from https://github.com/rmaestre/FastDTW

[30] Dr. T. H. Sreenivas Mandara Nagendra, C. N. Chinnaswamy. 2018. Robot framework: A boon for automation. *IJSDR | 3, 11* (2018), 1–4.

[31] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. 2008. OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review* 38, 2 (2008), 69–74.

[32] Medium. 2016. *What is Acceptable Jitter?* Retrieved from https://medium.com/@datapath_io/what-is-acceptable-jitter-7e93c1e68f9b

[33] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An ensemble of autoencoders for online network intrusion detection. *arXiv preprint arXiv:1802.09089* (2018).

[34] Saurav Nanda, Faheem Zafari, Casimer DeCusatis, Eric Wedaa, and Baijian Yang. 2016. Predicting network attack patterns in SDN using machine learning approach. In *2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE, 167–172.

[35] Ankur Kumar Nayak, Alex Reimers, Nick Feamster, and Russ Clark. 2009. Resonance: Dynamic access control for enterprise networks. In *Proceedings of the 1st ACM Workshop on Research on Enterprise Networking*. 11–18.

[36] Timothy Nelson, Christopher Barratt, Daniel J. Dougherty, Kathi Fisler, and Shriram Krishnamurthi. 2010. The Margrave tool for firewall analysis. In *Proceedings of the 24th Large Installation System Administration Conference (LISA 10)*.

[37] Huy Anh Nguyen, Tam Van Nguyen, Dong Il Kim, and Deokjai Choi. 2008. Network traffic anomalies detection and identification with flow monitoring. In *2008 5th IFIP International Conference on Wireless and Optical Communications Networks (WOCN'08)*. IEEE, 1–5.

[38] ONF. [n. d.]. *Interface Topology Service*. Retrieved from https://api.onosproject.org/2.7.0/apidocs/org/onosproject/net/topology/TopologyService.html

[39] Huijun Peng, Zhe Sun, Xuejian Zhao, Shuhua Tan, and Zhixin Sun. 2018. A detection method for anomaly flow in software defined network. *IEEE Access* 6 (2018), 27809–27817.
[40] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Joe Stringer, Pravin Shelar, et al. 2015. The design and implementation of open {vSwitch}. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 117–130.
[41] Stan Salvador and Philip Chan. 2007. Toward accurate dynamic time warping in linear time and space. *Intelligent Data Analysis* 11, 5 (2007), 561–580.
[42] Gustavo Frigo Scaranti, Luiz Fernando Carvalho, Sylvio Barbon Junior, Jaime Lloret, and Mario Lemes Proença Jr. 2022. Unsupervised online anomaly detection in software defined network environments. *Expert Systems with Applications* 191 (2022), 116225.
[43] Jaishree Singh, Hari Ram, and Dr J. S. Sodhi. 2013. Improving efficiency of apriori algorithm using transaction reduction. *International Journal of Scientific and Research Publications* 3, 1 (2013), 1–4.
[44] SpeedGuide.net. 2024. TCP/IP Ports and Protocols Database. Retrieved November 15, 2024 from https://www.speedguide.net/ports.php
[45] V. A. Stafford. 2020. Zero trust architecture. *NIST Special Publication* 800 (2020), 207.
[46] Alok Tongaonkar, Niranjan Inamdar, and R. Sekar. 2007. Inferring higher level policies from firewall rules. In *LISA*, Vol. 7. 1–10.
[47] Michael Tschannen, Olivier Bachem, and Mario Lucic. 2018. Recent advances in autoencoder-based representation learning. *arXiv preprint arXiv:1812.05069* (2018).
[48] Romans Vanickis, Paul Jacob, Sohelia Dehghanzadeh, and Brian Lee. 2018. Access control policy enforcement for zero-trust-networking. In *2018 29th Irish Signals and Systems Conference (ISSC)*. IEEE, 1–6.
[49] WIDE Project. 2023. MAWI Working Group Traffic Archive. https://mawi.wide.ad.jp/mawi/ Accessed: 2024-10-24.
[50] Lily Yang, Ram Dantu, Terry Anderson, and Ram Gopal. 2004. *Forwarding and Control Element Separation (ForCES) Framework*. Technical Report.
[51] Sultan Zavrak and Murat İskefiyeli. 2020. Anomaly-based intrusion detection from network flow features using variational autoencoder. *IEEE Access* 8 (2020), 108346–108358.

Received 20 November 2024; revised 20 November 2024; accepted 18 December 2024